Swift-FedGNN: Federated Graph Learning with Low Communication and Sample Complexities

Peiwen Qiu, Ziyue Luo, Ness B. Shroff, and Jia (Kevin) Liu
Dept. of Electrical and Computer Engineering
The Ohio State University
Columbus, OH, USA

Abstract

In recent years, there has been growing interest in federated graph learning (FGL), which enables privacy-preserving training over geographically distributed graph-structured data. However, in FGL, a node's representation may depend on neighbors located on other clients. Existing methods therefore make two unsatisfactory compromises: they either ignore cross-client neighbors, losing valuable graph signals, or repeatedly exchange cross-client information, incurring substantial communication and sampling overhead that can dominate runtime. This challenge is further exacerbated in mini-batch training on large or dense graphs. To address this challenge, we propose Swift-FedGNN, a communication-efficient mini-batch FGL framework for element-wise message-passing GNNs. Swift-FedGNN performs efficient parallel local training in most rounds and only periodically conducts cross-client training on a sampled subset of clients. During cross-client training, to avoid raw feature exchange, cross-client neighbor information is aggregated twice: first at the remote clients and then at the server before being sent to the training client. Theoretically, without assuming unbiased or consistent gradient estimators, we directly bound the gap between full and stochastic gradients under neighbor sampling and missing cross-client neighbors, and establish an $\mathcal{O}(T^{-1/2})$ convergence rate. This matches the rate of state-of-the-art sampling-based GNN methods despite operating in the more challenging federated setting, while striking a good balance between communication and accuracy trade-off. Experiments on real-world datasets show that Swift-FedGNN substantially improves efficiency while maintaining comparable accuracy. Such performance gains are even more pronounced on large and dense graphs.

1 Introduction

Graph neural networks (GNNs) have been widely used across various applications, such as social networks [1, 2], recommendation systems [3, 4], and drug discovery [5, 6]. In many of these applications, however, the underlying graph data cannot be stored in a single location due to its sheer size. Instead, these large graph data need to be partitioned into smaller subgraphs and stored in a geo-distributed fashion across organizations or sites, where the subgraphs cannot be freely shared between clients due to privacy, policy, or operational constraints. Federated learning (FL) [7] is therefore an attractive paradigm for learning over geo-distributed graph data, which gives rise to federated graph learning (FGL).

However, FGL is significantly harder to design and implement appropriately than the standard FL counterpart due to the key challenge of cross-client dependency. Specifically, a node's representation depends on its neighbors, some of which may reside on other clients, refer to as *"cross-client neighbors"* (shown as the dashed links between nodes in Figure 1). These cross-client neighbors create a fundamental dilemma in FGL. On the one hand, ignoring cross-client information loses

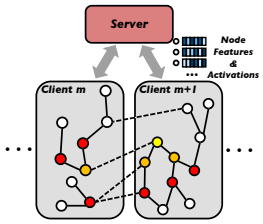


Figure 1: FGL setting. Dashed lines show graph dependency cross-clients.

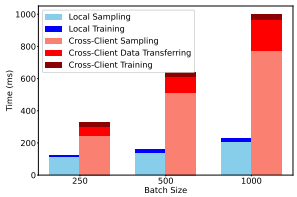


Figure 2: Per-iteration time breakdown: local vs. cross-client training.

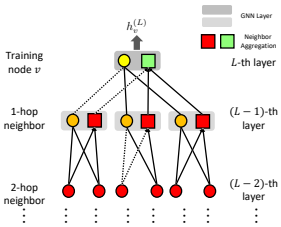


Figure 3: FGL model. Dashed lines show communication between clients.

graph signals and hurts accuracy [8]. On the other hand, fully incorporating them requires repeatedly sampling, retrieving, and communicating remote graph information in each training round, which can dominate the overall training cost. This challenge is further exacerbated in mini-batch training on large or dense graphs. As an example, the timing breakdown in Figure 2, obtained by training a two-layer GNN on the Amazon product co-purchasing dataset [9] under 1 Gbps network bandwidth, shows that cross-client sampling, data transfer, and cross-client training can dominate per-iteration runtime and make training *five times* slower than purely local updates.

Existing FGL approaches typically make two unsatisfactory compromises. Some methods ignore cross-client neighbors and accept the resulting information loss [10]. Others seek to preserve cross-client neighbor information [11–13], but do so at the cost of substantial cross-client data transfer, leading to high communication overhead, and additional storage of graph structure and node features for cross-client neighbors at each client. These requirements not only creates memory-intensive requirements, but may also potentially violate data privacy constraints. To date, what remains missing is an FGL design that preserves enough cross-client neighbor information to maintain accuracy, while avoiding these cross-client overheads (both communication and storage) that make cross-client training inefficient in practice (see Section 2 for a detailed discussion).

To address these challenges, we propose Swift-FedGNN, a mini-batch- and sampling-based framework for communication- and sample-efficient FGL. The *key idea* is that clients perform efficient parallel local training in most rounds using only local graph data, while only a sampled subset of clients periodically performs cross-client training that incorporates cross-client neighbors. This design substantially reduces communication and sampling overhead while limiting the information loss caused by missing cross-client neighbors. Moreover, during cross-client training, the cross-client neighbor information is aggregated twice: first at the remote clients and then at the server before being sent to the training client. As a result, Swift-FedGNN reduces communication and storage overhead while ensuring that only aggregated neighbor information, rather than raw node features, is exchanged, thereby enhancing privacy. Our design is deliberately scoped to element-wise message-passing GNNs (*e.g.*, GCN [14], GraphSAGE [15], GIN [16], and SGCN [17]), whose decomposable aggregation structure makes this two-stage compression possible. As an additional key insight that could be of general interest in FGL, in this paper, we show that non-element-wise GNNs (*e.g.*, attention-based GAT [18]), which require finer-grained interactions among *individual* neighbors, are inherently *not* ideal GNN model choices for FGL, where communication efficiency is of utmost importance.

However, the use of intermittent cross-client training in our Swift-FedGNN algorithm renders its design and performance analysis highly non-trivial because of two sources of bias resulted from: i) *neighbor-sampling* and ii) *omission of cross-client neighbors during local training*. In multilayer GNNs, these errors propagate through repeated alternations of neighbor aggregation and nonlinear transformation, making the convergence analysis substantially more challenging than in deep neural networks (DNNs). Unlike prior works that rely on strong assumptions on stochastic gradient bias (e.g., unbiased [19] or consistent [20] gradients), we *directly* bound the stochastic gradient approximation errors, yielding a more realistic analysis for federated GNN training. A key finding of this analysis is that the *biased* stochastic gradients in GNNs, arising from missing cross-client neighbors and neighbor sampling, are *positively correlated* with the network depth, reflecting a challenge specific to multilayer GNNs in the federated setting. Building on this foundation, we obtain an $\mathcal{O}(T^{-1/2})$ convergence rate with a residual term that explicitly captures the trade-off among cross-client training frequency, sampled clients, and information loss. This rate matches the rate of state-of-the-art (SOTA) sampling-based GNN methods, despite operating in the more challenging federated setting.

Our experiments show that Swift-FedGNN is particularly effective on large or dense graphs, where Swift-FedGNN converges faster in wall-clock time than representative baselines while maintaining comparable accuracy. It also consistently reduces the communication cost required to reach a target accuracy on most datasets, performs effectively under random partitioning, and scales favorably to large-scale federated settings. Collectively, the theoretical and empirical results convey a consistent message: *Aggressive per-round cross-client training is often unnecessary, and sparse cross-client training can recover much of its benefit at substantially lower cost.* In summary, our main contributions are as follows:

- We develop Swift-FedGNN, a communication and sample efficient mini-batch framework for federated training of element-wise message-passing GNNs, built around periodic cross-client training and aggregated cross-client neighbor communication.
- We provide a rigorous convergence analysis for Swift-FedGNN by directly bounding gradient approximation errors caused by neighbor sampling and missing cross-client neighbors. This yields an $\mathcal{O}(T^{-1/2})$ convergence rate for Swift-FedGNN, matching the SOTA rate of sampling-based GNN methods, while explicitly exposing an communication–accuracy trade-off.
- Extensive experiments on real-world graph datasets validate that Swift-FedGNN substantially improves communication efficiency, while maintaining competitive accuracy, with the largest gains on large or dense graphs.

2 Related Work

In this section, we provide an overview on DGL and offer a comprehensive comparison with the most relevant work on FGL.

1) Distributed Graph Learning: Distributed graph learning (DGL) frameworks (*e.g.*, DistDGL [21], Pytorch Geometric [22], AliGraph [23] and Dorylus [24]) have been developed to train large-scale graph datasets via cross-device sampling and direct worker-to-worker communication, and often spend up to 80% of the total training time on data communication [25]. Although various optimizations (graph partitioning [21], caching [26], communication strategies [27], and parallel training [25, 28]) have been proposed to expedite DGL, they commonly require direct data sharing between workers. Unfortunately, such requirements are not suitable for the FGL settings, where raw graph data must remain local.

LLCG [29] is the closest DGL framework that avoids direct node-feature transfer between workers, making it potentially applicable to the FGL setting. In LLCG, each worker trains only on its local graph partition. To address missing cross-device neighbor information, LLCG employs a central server to periodically perform full-neighbor training with neighbor aggregation across all workers. However, this approach imposes significant communication overhead on the server, which needs to communicate with every worker to perform the full-neighbor training.

2) Federated Graph Learning: Existing FGL methods differ mainly in how they handle cross-client neighbors. One line of work treats client graphs as independent and ignores cross-client neighbors [10], which avoids communication but loses graph signals, fails to reflect real-world scenarios, and can degrade the performance of the trained model. Another line of work assumes that the clients' local graphs have overlapped nodes and the edges are distributed [30], which may not be true in many real-world situations. The third line of work explicitly preserves cross-client neighbor information, which is highly communication-intensive. For example, the method in [11] exchanges cross-client neighbor information in every training round, incurring considerable communication overhead and exposes private node information to other clients. While the method in [13] employs a one-time exchange of full cross-client neighbor information prior to training, it relies on full-graph training and causes significant per-client memory overhead, making it impractical for large-scale graphs. Also, adapting it to sampling-based FGL would require per-iteration cross-client exchanges (since each mini-batch involves different training nodes and sampled neighbors), further exacerbating communication overhead.

The most related work to ours is [12], which uses sparse cross-client neighbor sampling to supplement the lost information of the cross-client neighbors and reduce the communication overhead. In [12], each client periodically samples and exchanges these neighbors with other clients, reusing the most recent sampled neighbors in between exchanges. However, as training progresses, the frequency

of information exchange increases, leading to higher communication costs. Furthermore, direct client-to-client data transfers and caching is allowed in [12], which could lead to privacy concerns and additional storage overhead. Also, repeatedly reusing the same neighbor data in [12] introduces bias that degrades the performance.

In contrast, our Swift-FedGNN algorithm adopts a drastically different design: clients perform local training in most rounds, and only a sampled subset of clients periodically incorporates cross-client neighbors. This design reduces both communication and storage overhead. Moreover, cross-client neighbor information is aggregated *twice,* first at the remote clients and then at the server. Thus, *training clients receive only aggregated neighbor information rather than raw node features,* which further lowers communication cost and helps preserve data privacy.

3 Federated Graph Learning: Preliminaries

In this section, we provide the background of the mathematical formulation for training GNNs in a federated setting. Consider a graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is a set of nodes with $N = |\mathcal{V}|$ and $\mathcal{E}$ is a set of edges. We consider a standard federated setting that has a central server and a set of $\mathcal{M}$ clients with $M = |\mathcal{M}|$. The graph $\mathcal{G}$ is partitioned and geographically distributed over these clients, and each client m contains a local *subgraph* represented by $\mathcal{G}^m(\mathcal{V}^m, \mathcal{E}^m)$. Note that $\bigcup_{m=1}^{M} \mathcal{G}^m \neq \mathcal{G}$ due to the missing cross-client edges between clients ($\bigcup_{m=1}^{M} \mathcal{E}^m \neq \mathcal{E}$). In addition, we assume that the nodes are *disjointly* partitioned across clients, *i.e.*, $\bigcup_{m=1}^{M} \mathcal{V}^m = \mathcal{V}$ and $\bigcap_{m=1}^{M} \mathcal{V}^m = \emptyset$. Each node $v \in \mathcal{V}^m$ has a feature vector $\boldsymbol{x}_v^m \in \mathbb{R}^d$, and each training node $v \in \mathcal{V}_{train}^m$ corresponds to a label y_v^m, where $\mathcal{V}_{train}^m \subseteq \mathcal{V}^m$.

In FGL, the clients collaboratively train a model on distributed graph data under the coordination of the central server. The goal of FGL is to solve the following problem:

$$\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) := \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} F^m(\boldsymbol{\theta}) := \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} \frac{1}{|\mathcal{V}_B^m|} \sum_{v \in \mathcal{V}_B^m} \ell^m\left(\boldsymbol{h}_v^{(L),m}, y_v^m\right), \tag{1}$$

where ℓ^m is a loss function (*e.g.*, cross-entropy loss) at client m, $\boldsymbol{h}_v^{(L),m}$ is the L-th layer embedding of node v, $\mathcal{V}_B^m$ denotes a mini-batch of training nodes uniformly sampled from $\mathcal{V}_{train}^m$, and $\boldsymbol{\theta} := \left\{\boldsymbol{W}^{(l)}\right\}_{l=1}^{L}$ corresponds to all model parameters. Note that, unlike the data parallelism in standard FL (*i.e.*, each client has a full copy of the model and a subset of data), FGL is based on **model parallelism**, where each client possesses a subgraph of the GNN model and the corresponding features of all graph data samples.

In this work, we consider message-passing GNNs, where each node representation (embedding) in the graph is updated element-wise by aggregating information from its neighboring nodes and then applying a learnable transformation. Note that the neighbors of a node v may be located locally or on some remote clients. We let $m(v)$ and $\bar{\mathcal{M}}(v)$ represent the local client and the set of remote clients that host node v's neighbors. Clearly, $\bar{\mathcal{M}}(v) \subseteq \mathcal{M} \setminus \{m(v)\}$. We let $\mathcal{N}^{m(v)}(v)$ and $\mathcal{N}^{\bar{m}(v)}(v)$ denote the sets of node v's neighbors on its local client $m(v)$ and remote clients $\bar{m}(v) \in \bar{\mathcal{M}}(v)$, respectively. As shown in Figure 3, to compute the embedding of node v at the l-th layer in a GNN with L layers, the client $m(v)$ first aggregates the neighbor information from both itself and the remote clients $\bar{m}(v)$, and then updates the embedding of node v as follows:

$$\boldsymbol{h}_{\mathcal{N}(v)}^{(l)} = \mathrm{AGG}\Bigg(\underbrace{\left\{\boldsymbol{h}_u^{(l-1),m(v)} \mid u \in \mathcal{N}^{m(v)}(v)\right\}}_{\text{local}} \cup \underbrace{\left\{\bigcup_{\bar{m}(v) \in \bar{\mathcal{M}}(v)} \left\{\boldsymbol{h}_u^{(l-1),\bar{m}(v)} \mid u \in \mathcal{N}^{\bar{m}(v)}(v)\right\}\right\}}_{\text{remote}}\Bigg),$$

$$\boldsymbol{h}_v^{(l),m(v)} = \sigma\left(\boldsymbol{W}^{(l)} \cdot \mathrm{COMB}\left(\boldsymbol{h}_v^{(l-1),m(v)}, \boldsymbol{h}_{\mathcal{N}(v)}^{(l)}\right)\right), \tag{2}$$

where $\boldsymbol{h}_{\mathcal{N}(v)}^{(l)}$ is the aggregated embedding from node v's neighbors at layer l, $\boldsymbol{h}_v^{(l),m(v)}$ is the embedding of node v located on client $m(v)$ and is initialized as $\boldsymbol{h}_v^{(0),m(v)} = \boldsymbol{x}_v^{m(v)}$, $\boldsymbol{W}^{(l)}$ represents the weight matrix at l-th layer, $\sigma(\cdot)$ corresponds to an activation function (*e.g.*, ReLU), $\mathrm{AGG}(\cdot)$ is an aggregation function (*e.g.*, mean, sum, or max), and $\mathrm{COMB}(\cdot)$ is a combination function (*e.g.*, concatenation).

Algorithm 1: Swift-FedGNN Algorithm.

Input: Initial parameters $\boldsymbol{\theta}_0$, learning rate α, cross-client correction frequency I, and sampled-client budget K

for $t = 0$ ***to*** $T-1$ **do**
  if t *mod* $I = 0$ **then**
    Randomly sample $|\mathcal{K}|$ clients
    for $m \in \mathcal{M}$ ***in parallel*** **do**
      if $m \in \mathcal{K}$ **then**
        Client update with local graph and cross-client neighbors using Algorithm 2
      else
        Client update with local graph using Algorithm 3
  else
    for $m \in \mathcal{M}$ ***in parallel*** **do**
      Client update with local graph using Algorithm 3
  Server:
  Aggregate and update global model parameter as: $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} \nabla \widetilde{F}^m \left(\boldsymbol{\theta}_t^m\right)$

Algorithm 2: Client m at iteration t: update with local graph and cross-client neighbors (cross-client training).

Receive global parameter $\boldsymbol{\theta}_t^m = \boldsymbol{\theta}_t$
Construct a mini-batch $\mathcal{B}_v^m$ of training nodes
Server samples a subset of L-hop neighbors $\mathcal{S} = \left\{\mathcal{S}^{(l)}\right\}_{l=0}^{L-1}$ for the nodes in $\mathcal{B}_v^m$

for $l = 1$ ***to*** L **do**
  `/* Derive` l`-th layer embedding of node` $v \in \mathcal{B}_v^m$ `if` $l = L$`, otherwise` $v \in \mathcal{S}^{(l)}$ `*/`
  for *Remote client* $\bar{m}(v) \in \bar{\mathcal{M}}(v)$ ***in parallel*** **do**
    Aggregate the neighbor embeddings using Eq. (5)
    Send the aggregated embedding $\boldsymbol{h}_{\mathcal{N}(v)}^{(l),\bar{m}(v)}$ to server
  Server:
  Aggregate the neighbor embeddings from the remote clients using Eq. (6)
  Send the aggregated cross-client neighbor embedding $\boldsymbol{r}_{\mathcal{N}(v)}^{(l)}$ to Client $m(v)$
  Client $m(v)$**:** Compute node embeddings using Eq. (7) and (8)

Compute stochastic gradient $\nabla \widetilde{F}^m \left(\boldsymbol{\theta}_t^m\right)$ and send to server

Algorithm 3: Client m at iteration t: update with local graph (local training).

Receive global parameter $\boldsymbol{\theta}_t^m = \boldsymbol{\theta}_t$
Construct a mini-batch $\mathcal{B}_v^m$ of training nodes
Sample a subset of L-hop neighbors $\mathcal{S} = \left\{\mathcal{S}^{(l)}\right\}_{l=0}^{L-1}$ for the nodes in $\mathcal{B}_v^m$

for $l = 1$ ***to*** L **do**
  `/* Derive` l`-th layer embedding of node` $v \in \mathcal{B}_v^m$ `if` $l = L$`, otherwise` $v \in \mathcal{S}^{(l)}$ `*/`
  Compute node embeddings using Eq. (3) and (4)

Compute stochastic gradient $\nabla \widetilde{F}^m \left(\boldsymbol{\theta}_t^m\right)$ and send to server

Compared with DGL (cf. Section 2), the *key challenge* in FGL is that clients *cannot* freely exchange raw node features or local graph structure due to privacy restrictions. Consequently, care must be taken when computing the update in Eq. equation 2 when part of a node's neighbors resides on other clients.

4 The Swift-FedGNN Algorithm

In this section, we present our proposed FGL framework called Swift-FedGNN for solving Problem (1) with low sampling and communication costs. We start with the algorithmic design of Swift-FedGNN in Section 4.1, which is followed by a discussion on FGL model choices in Section 4.2 to reveal further insights for FGL.

4.1 The Algorithmic Design of Swift-FedGNN

A) Basic Idea: The main framework of Swift-FedGNN is shown in Algorithm 1. The basic idea of Swift-FedGNN is that, rather than performing cross-client training in every round at each client, the clients in Swift-FedGNN primarily conduct efficient *local training in parallel*, and a set of randomly selected clients *periodically* carry out the time-consuming cross-client training. By offloading part of the graph operations involving cross-client neighbor aggregation to the server and remote clients,

Swift-FedGNN eliminates the need for sharing raw graph features among clients and reduces both the frequency and volume of cross-client neighbor communication.

Specifically, for every $I-1$ consecutive iterations, all clients perform parallel local training using their own subgraphs. At every I-th iteration, a subset of clients $\mathcal{K} \subseteq \mathcal{M}$ with $|\mathcal{K}| = K$ is sampled to perform cross-client training, while the remaining clients continue with local training. Thus, I controls how often cross-client correction is performed, while K controls how many clients receive this correction in each such round. Both parameters expose the central communication–accuracy trade-off in Swift-FedGNN. In what follows, we will describe each key step of Swift-FedGNN in further details.

B) Local Training: In the t-th local training iteration, every client m updates the local GNN model using only its local graph, as shown in Algorithm 3. Specifically, client m samples a mini-batch of training nodes $\mathcal{B}_v^m$ and a subset of L-hop neighbors for these training nodes, denoted as $\mathcal{S} = \left\{\mathcal{S}^{(l)}\right\}_{l=0}^{L-1}$, all drawn from the local graph data. To compute the embedding of node v in the l-th GNN layer ($v \in \mathcal{B}_v^m$ if $l = L$, otherwise $v \in \mathcal{S}^{(l)}$), client m first conducts the neighbor aggregation based on the sampled neighbors using:

$$\boldsymbol{h}_{\mathcal{N}(v)}^{(l)} = \mathrm{AGG}\left(\left\{\boldsymbol{h}_u^{(l-1),m} \mid u \in \widetilde{\mathcal{N}}^m(v)\right\}\right), \tag{3}$$

where $\widetilde{\mathcal{N}}^m(v)$ denotes a set of sampled neighbors located on client m for node v, with $\widetilde{\mathcal{N}}^m(v) \subseteq \mathcal{S}^{(l-1)}$ and $\widetilde{\mathcal{N}}^m(v) \subseteq \mathcal{N}^m(v)$. Then, client m updates the embedding of node v in the l-th GNN layer based on the aggregated neighbor information and the embedding of node v from the $(l-1)$-th layer, as follows:

$$\boldsymbol{h}_v^{(l),m} = \sigma\left(\boldsymbol{W}_t^{(l),m} \cdot \mathrm{COMB}\left(\boldsymbol{h}_v^{(l-1),m}, \boldsymbol{h}_{\mathcal{N}(v)}^{(l)}\right)\right). \tag{4}$$

These local training rounds require no cross-client neighbor communication and can be executed fully in parallel.

C) Periodic Cross-client Training: At every I-th iteration, a set of K clients, uniformly sampled from $\mathcal{M}$, conducts cross-client training on the local GNN models using both their local and cross-client neighbors' graph data, as shown in Algorithm 2. Rather than directly exchanging raw node features between clients, Swift-FedGNN partitions the GNN training across the training client, the remote clients, and the server. The server coordinates the sampling of relevant cross-client neighbors. Each remote client locally aggregates its sampled neighbors (*i.e.*, node features or intermediate activations at each GNN layer), and the server aggregates such cross-client neighbor information again. The training client then combines the resulting cross-client neighbor information with its local neighbor information. This design avoids exchanging raw node features and substantially reduces both communication and storage costs. Operations performed on the server and the remote clients are colored using `server` and `remote client`, respectively.

Specifically, client $m \in \mathcal{K}$ samples a mini-batch of training nodes $\mathcal{B}_v^m$. Then, with the cooperation of the server, a subset of L-hop neighbors for the training nodes in $\mathcal{B}_v^m$ is sampled and represented as $\mathcal{S} = \left\{\mathcal{S}^{(l)}\right\}_{l=0}^{L-1}$. The nodes $v \in \mathcal{B}_v^m$ are located on client m, while for $v \in \mathcal{S}^{(l)}$ with $l < L$, the nodes may be located on clients other than m, denoting the client storing v as $m(v)$. Recall that $\bar{\mathcal{M}}(v) \subseteq \mathcal{M} \backslash \{m(v)\}$ denotes the set of remote clients that host node v's sampled cross-client neighbors. Each remote client $\bar{m}(v) \in \bar{\mathcal{M}}(v)$ may host multiple sampled neighbors of the training node v, and the numbers of such neighbors can vary across remote clients.

D) Algorithmic Description of Cross-Client Training: Computing the l-th layer embedding of node v in Swift-FedGNN consists of four steps. Steps 1 to 3 below are used to aggregate the neighbor information of node v, and Step 4 is used to update the node v's embedding at l-th GNN layer.

Step 1) Remote-client-side Aggregation: Each remote client $\bar{m}(v)$ aggregates its sampled neighbors of node v in *parallel,* using

$$\boldsymbol{h}_{\mathcal{N}(v)}^{(l),\bar{m}(v)} = \mathrm{AGG}\left(\left\{\boldsymbol{h}_u^{(l-1),\bar{m}(v)} \mid u \in \widetilde{\mathcal{N}}^{\bar{m}(v)}(v)\right\}\right). \tag{5}$$

Only the aggregated results from each remote client $\bar{m}(v)$ are sent to the server. This is the *first* point at which communication is compressed: the remote client does not send raw node features or per-neighbor embeddings.

Step 2) Server-side Aggregation: Upon the reception of the aggregated neighbor information from all remote clients in $\bar{\mathcal{M}}(v)$, the server aggregates this information again as follows rather than simply forwarding it to client $m(v)$:

$$\boldsymbol{r}_{\mathcal{N}(v)}^{(l)} = \mathrm{AGG}\left(\left\{\boldsymbol{h}_{\mathcal{N}(v)}^{(l),\bar{m}(v)} \mid \bar{m}(v) \in \bar{\mathcal{M}}(v)\right\}\right). \tag{6}$$

This *second* aggregation further reduces communication by minimizing the amount of data transmitted between clients and the server, while avoiding the direct exchange of raw neighboring node information. Note that, if differential privacy guarantees are desired, Swift-FedGNN is compatible with various differentially privacy noise injection techniques and federated encryption protocols.

Step 3) Fusion of Local and Cross-client Neighbors: The neighbor information for node v, including both sampled local neighbors and sampled cross-client neighbors, is aggregated as follows:

$$\boldsymbol{h}_{\mathcal{N}(v)}^{(l)} = \mathrm{AGG}\Bigg(\underbrace{\left\{\boldsymbol{h}_{u}^{(l-1),m(v)} \mid u \in \widetilde{\mathcal{N}}^{m(v)}(v)\right\}}_{\text{local}} \bigcup \underbrace{\left\{\boldsymbol{r}_{\mathcal{N}(v)}^{(l)}\right\}}_{\text{remote}}\Bigg). \tag{7}$$

The cross-client neighbor information used here helps mitigate information loss and reduce performance degradation caused by connected nodes being distributed across different clients.

Step 4) Node Update: The embedding of node v in the l-th GNN layer is updated using the aggregated neighbor information and the embedding of node v from the $(l-1)$-th layer as follows:

$$\boldsymbol{h}_{v}^{(l),m(v)} = \sigma\left(\boldsymbol{W}_{t}^{(l),m(v)} \cdot \mathrm{COMB}\big(\boldsymbol{h}_{v}^{(l-1),m(v)}, \boldsymbol{h}_{\mathcal{N}(v)}^{(l)}\big)\right). \tag{8}$$

Once the embeddings of the training nodes in the mini-batch have been computed, client m evaluates the stochastic gradient $\nabla\widetilde{F}^{m}\left(\boldsymbol{\theta}_{t}^{m}\right)$ and sends it to the server. The server then aggregates the gradients from all clients and updates the global model according to $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_{t} - \alpha\frac{1}{|\mathcal{M}|}\sum_{m\in\mathcal{M}}\nabla\widetilde{F}^{m}\left(\boldsymbol{\theta}_{t}^{m}\right)$, where α is the learning rate.

4.2 Discussions: Model Choices in FGL

In designing our communication-efficient Swift-FedGNN, we deliberately focus on GNNs with *element-wise* aggregations (*e.g.*, GCN [14], GraphSAGE [15], GIN [16], SGCN [17]) rather than GNNs with non-element-wise aggregations, although some GNN models in the latter class are powerful (*e.g.*, attention-based GAT [18]). The **fundamental rationale** behind this choice is that element-wise aggregators (*e.g.*, mean, sum, max) enable cross-client neighbor information to be aggregated in a *two-stage* manner at remote clients and the server before transmissions (cf. Steps 1-2 in Swift-FedGNN), thereby avoiding the communication of raw node features or individual neighbor embeddings and significantly reducing communications. However, such raw-node-feature-free aggregations are *impossible* for GNNs with non-element-wise aggregators, which renders them *non-ideal* for FGL, where communication efficiency is of utmost importance. In what follows, we provide an in-depth discussion that offers insights of broader interest in FGL.

1) GNNs with Element-wise Aggregations: Assume that each GNN layer uses a uniform neighbor sampling fan-out of F. Then, the worst-case number of sampled neighbors per training node at layer $l \in [1, L]$ is F^{l}, which grows exponentially with l. Let $p_{(l)} \in (0,1)$ be the fraction of neighbors at layer l located on remote clients. If the $p_{(l)}F^{l}$ cross-client neighbors at layer l are distributed across $C_{(l)} < M$ remote clients, then the total communication cost of Swift-FedGNN over T training iterations for exchanging aggregated cross-client neighbor embeddings is $\mathcal{O}\big(\frac{T}{I}KB\sum_{l=1}^{L}C_{(l)}d_{(l-1)}^{emb}\big)$, where B is the batch size per client, and $d_{(l)}^{\text{emb}}$ is the embedding dimension at layer l.

The analysis above also highlights the communication efficiency of Swift-FedGNN stemming from four key factors: **a) Periodic** cross-client training performed once every I iterations; **b)** $K < M$ **client sampling** in each cross-client training iteration; **c) Two-stage aggregation** to compress information prior to transmission (*i.e.*, $C_{(l)} \ll p_{(l)}F^{l}$); and **d) Lower-dimensional** hidden embedding transmissions (*i.e.*, $d_{(l-1)}^{\text{emb}} < d_{(0)}^{\text{emb}}$ for $l \geq 2$).

2) GNNs with Non-element-wise Aggregations: In GNNs with non-element-wise aggregations, nodes require access to *individual* neighbor features or embeddings to perform nonlinear operations

(*e.g.*, attention in GAT), which prevents pre-aggregation and leads to substantially higher communication overhead. To see this, consider GAT as a representative example. Let all GNN layers have the same embedding dimension as those in GNNs with element-wise aggregation. Under the same setting, without the benefit of two-stage aggregation as in Swift-FedGNN, incorporating GAT incurs a communication cost of $\mathcal{O}\big(\frac{T}{I}KB\sum_{l=1}^{L}p_{(l)}F^l d_{(l-1)}^{emb}\big)$, greater than that of element-wise GNNs by a factor of $\frac{\sum_{l=1}^{L}p_{(l)}F^l}{\sum_{l=1}^{L}C_{(l)}}$. Moreover, since $p_{(l)}F^l \gg C_{(l)}$, and F^l grows exponentially with the number of layers, the total communication cost of GAT could be prohibitive.

5 Theoretical Performance Analysis

In this section, we establish the theoretical convergence guarantees for Swift-FedGNN using Graph Convolutional Network (GCN) [14] as the GNN architecture to solve Problem (1). The convergence guarantees of GCN can also be extended to other element-wise operation-based GNNs (*e.g.*, GraphSAGE and GIN).

1) Preliminaries: We start with providing necessary backgrounds on the GCN model. For a graph $\mathcal{G}$, the structure can be represented by its adjacency matrix $\boldsymbol{A} \in \mathbb{R}^{N\times N}$, where $\boldsymbol{A}_{vu}=1$ if $(v,u)\in\mathcal{E}$, otherwise $\boldsymbol{A}_{vu}=0$. Define $\hat{\boldsymbol{A}}=\boldsymbol{A}+\boldsymbol{I}$, and let $\boldsymbol{D}\in\mathbb{R}^{N\times N}$ be the degree matrix with $\boldsymbol{D}_{vv}=\sum_u \hat{\boldsymbol{A}}_{vu}$. The corresponding propagation matrix, which determines how information is passed between nodes, can be computed as $\boldsymbol{P}=\boldsymbol{D}^{-1/2}\hat{\boldsymbol{A}}\boldsymbol{D}^{-1/2}$. For the local subgraph $\mathcal{G}^m$ located on client m, the adjacency matrix $\boldsymbol{A}^m$ can be decomposed as $\boldsymbol{A}^m=\boldsymbol{A}^m_{local}+\boldsymbol{A}^m_{remote}$, where $\boldsymbol{A}^m_{local}$ corresponds to the nodes located on client m, and $\boldsymbol{A}^m_{remote}$ corresponds to their cross-client neighbors located on the remote clients other than m. Then, the propagation matrix can be calculated as $\boldsymbol{P}^m=\boldsymbol{D}_m^{-1/2}\left(\boldsymbol{A}^m+\boldsymbol{I}^m\right)\boldsymbol{D}_m^{-1/2}$, and can be further decomposed as $\boldsymbol{P}^m=\boldsymbol{P}^m_{local}+\boldsymbol{P}^m_{remote}$, where $\boldsymbol{P}^m_{local}=\boldsymbol{D}_m^{-1/2}\left(\boldsymbol{A}^m_{local}+\boldsymbol{I}^m\right)\boldsymbol{D}_m^{-1/2}$ and $\boldsymbol{P}^m_{remote}=\boldsymbol{D}_m^{-1/2}\boldsymbol{A}^m_{remote}\boldsymbol{D}_m^{-1/2}$.

With the above matrix notations, the local training update in Eq. (3) and (4) can be written as $\widetilde{\boldsymbol{H}}_t^{(l),m}=\sigma\big(\widetilde{\boldsymbol{P}}^{(l),m}_{local}\widetilde{\boldsymbol{H}}^{(l-1),m}_{local}\boldsymbol{W}_t^{(l),m}\big)$. Similarly, the cross-client training update in Eq. (5)–(8) can be written as $\widetilde{\boldsymbol{H}}_t^{(l),m}=\sigma\big(\big(\widetilde{\boldsymbol{P}}^{(l),m}_{local}\widetilde{\boldsymbol{H}}^{(l-1),m}_{local}+\widetilde{\boldsymbol{P}}^{(l),m}_{remote}\widetilde{\boldsymbol{H}}^{(l-1),m}_{remote}\big)\boldsymbol{W}_t^{(l),m}\big)$. Next, we state several needed assumptions for our theoretical analysis.

Assumption 5.1. The loss function $\ell^m(\cdot,\cdot)$ is C_l-Lipschitz continuous and L_l-smooth in node embeddings $\boldsymbol{h}^{(L)}$, *i.e.*, there exist constants $C_l, L_l>0$ such that $\|\ell^m(\boldsymbol{h}_1^{(L)},y)-\ell^m(\boldsymbol{h}_2^{(L)},y)\|_2\le C_l\|\boldsymbol{h}_1^{(L)}-\boldsymbol{h}_2^{(L)}\|_2$ and $\|\nabla\ell^m(\boldsymbol{h}_1^{(L)},y)-\nabla\ell^m(\boldsymbol{h}_2^{(L)},y)\|_2\le L_l\|\boldsymbol{h}_1^{(L)}-\boldsymbol{h}_2^{(L)}\|_2$.

Assumption 5.2. The activation function $\sigma(\cdot)$ is C_σ-Lipschitz continuous and L_σ-smooth, *i.e.*, there exists constants $C_l, L_l>0$ such that $\|\sigma(\boldsymbol{z}_1^{(l)})-\sigma(\boldsymbol{z}_2^{(l)})\|_2\le C_\sigma\|\boldsymbol{z}_1^{(l)}-\boldsymbol{z}_2^{(l)}\|_2$ and $\|\nabla\sigma(\boldsymbol{z}_1^{(l)})-\nabla\sigma(\boldsymbol{z}_2^{(l)})\|_2\le L_\sigma\|\boldsymbol{z}_1^{(l)}-\boldsymbol{z}_2^{(l)}\|_2$.

Assumption 5.3. For any $l\in[L]$, the norm of weight matrices, the propagation matrix, and the node feature matrix are bounded by B_W, B_P and B_X, respectively, *i.e.*, $\|\boldsymbol{W}^{(l)}\|_F\le B_W$, $\|\boldsymbol{P}\|_F\le B_P$, and $\|\boldsymbol{X}\|_F\le B_X$. Note that this assumption is commonly used in the analysis of GNNs, *e.g.*, [19, 28, 31, 32].

2) Convergence Analysis: We first bound the errors between the full and stochastic gradients, which are caused by training-node sampling and neighbor sampling. To this end, we let $\nabla F^m_{full}(\boldsymbol{\theta}^m)$ and $\nabla\widetilde{F}^m_{full}(\boldsymbol{\theta}^m)$ denote the corresponding gradients when both local graph data and cross-client neighbors are used, respectively. Also, we let $\nabla F^m_{local}(\boldsymbol{\theta}^m)$ and $\nabla\widetilde{F}^m_{local}(\boldsymbol{\theta}^m)$ denote the full and stochastic gradients computed using only local graph data, respectively. Then, we have following result:

Lemma 5.4. *Under Assumptions 5.1–5.3, it holds that* $\|\nabla F^m_{local}(\boldsymbol{\theta}^m)-\nabla\tilde{F}^m_{local}(\boldsymbol{\theta}^m)\|_F\le LB^l_{\Delta G}$ *and* $\|\nabla F^m_{full}(\boldsymbol{\theta}^m)-\nabla\tilde{F}^m_{full}(\boldsymbol{\theta}^m)\|_F\le LB^f_{\Delta G}$*, where* $B^l_{\Delta G}$ *and* $B^f_{\Delta G}$ *are error bounds defined in Eq. (9) and (10) in Appendix C.*

The second source of error is specific to Swift-FedGNN: during local training, client m omits cross-client neighbors. The next lemma bounds the resulting gap between the full gradients computed with and without cross-client neighbors.

Lemma 5.5. *Under Assumptions 5.1–5.3, it holds that* $\|\nabla F_{full}^m(\boldsymbol{\theta}^m) - \nabla F_{local}^m(\boldsymbol{\theta}^m)\|_F \leq LB_{\Delta G}^r$, *where* $B_{\Delta G}^r$ *is the error bound defined in Eq. (11) in Appendix C.*

The proofs of Lemmas 5.4 and 5.5 are provided in Appendix D.6. Lemmas 5.4 and 5.5 indicate that these error terms grow with network depth L. This behavior reflects the layered structure of GNNs: each layer alternates neighbor aggregation with nonlinear transformation, so gradient approximation errors introduced at earlier layers propagate through subsequent layers. In the federated setting, this effect is further amplified by the intermittently missing cross-client neighbors. Based on these two lemmas, we now state the main convergence result of Swift-FedGNN.

Theorem 5.6. *Under Assumptions 5.1–5.3, choose the step-size* $\alpha = \min\left\{\frac{\sqrt{M}}{\sqrt{T}}, \frac{1}{L_F}\right\}$, *where* L_F *is the smoothness constant given in Lemma D.2. Then, the output of* Swift-FedGNN *applied to an* L*-layer GNN satisfies:*

$$\frac{1}{T}\sum_{t=0}^{T-1}\|\nabla\mathcal{L}(\boldsymbol{\theta}_t)\|^2 \leq \frac{2\left(\mathcal{L}(\boldsymbol{\theta}_0) - \mathcal{L}(\boldsymbol{\theta}^*)\right)}{\sqrt{MT}} + L^2\left(B_{\Delta G}^l + B_{\Delta G}^r\right)^2 + \frac{KL^2}{IM}\left(\left(B_{\Delta G}^f\right)^2 - \left(B_{\Delta G}^l + B_{\Delta G}^r\right)^2\right).$$

The proof sketch of Theorem 5.6 is presented in Appendix C, and the full proof is given in Appendix D. Theorem 5.6 shows that the convergence rate of Swift-FedGNN is $\mathcal{O}(T^{-1/2})$ to a neighborhood of the exact solution, which *matches* the SOTA convergence rate of sampling-based GNN algorithms, *e.g.*, [12, 19, 29, 32], even though Swift-FedGNN operates in the more challenging federated setting.

Three important remarks on Theorem 5.6 are in order: (1) When choosing $I = 1$ and $K = M$, Swift-FedGNN performs fully cross-client training, ensuring no information loss in the graph data. In this scenario, Swift-FedGNN experiences minimal residual error. Such error is caused by sampling and is inevitable. However, Swift-FedGNN suffers from maximum sampling and communication overhead; (2) When choosing $K = 0$, Swift-FedGNN conducts fully local training, resulting in the information loss of all the cross-client neighbors. Consequently, Swift-FedGNN encounters maximum residual error. Nonetheless, the sampling and communication overhead is minimized; and (3) The last term of the convergence rate bound in Theorem 5.6 is negative. Hence, decreasing I or increasing K would decrease the residual error due to less information loss of the cross-client neighbors at the cost of higher sampling and communication costs.

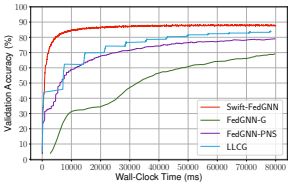
6 Numerical Experiments

In this section, we evaluate Swift-FedGNN on real-world datasets in terms of convergence, communication efficiency, communication-accuracy trade-off, and robustness to larger-scale and heterogeneous settings. Additional experimental details and results are provided in Appendix B, including further evidence for the theoretical findings, evaluations on another widely used GNN model, GIN [16], and additional validation-accuracy comparisons.

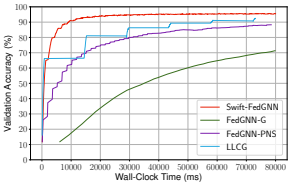
1) Experiment Settings: We train a representative GNN model, GraphSAGE [15], in the federated settings on five real-world node classification datasets: 1) ogbn-products [33]; 2) Reddit [15]; 3) ogbn-arxiv [33]; 4) flickr [34]; and 5) citeseer [35]. Among these, ogbn-products is the *largest* dataset commonly used in the FGL literature, while Reddit is known for its *high density.* These two datasets are thus informative for evaluating the communication bottleneck. Unless otherwise stated, all graphs are partitioned using METIS [36]. We also evaluate Swift-FedGNN on randomly partitioned graph data on the ogbn-products dataset, which induces more heterogeneous (non-i.i.d.) federated settings.

In our federated simulations, we use 20 clients for ogbn-products and 10 clients for all other datasets. For Swift-FedGNN, we set $K = 10$ for ogbn-products and $K = 5$ for all other datasets. We choose $I = 5$ for citeseer and $I = 10$ for the remaining datasets.

2) Baselines: Since Swift-FedGNN aims to reduce sampling and communication costs, we compare it with the most relevant algorithms that mitigate the information loss of cross-client neighbors through periodical (sampling-based) full-neighbor training: 1) LLCG [29]: A DGL framework that performs local training on each client independently, with periodic full-neighbor training conducted on a



(a) ogbn-products.



(b) Reddit.

Figure 4: Wall-clock convergence in validation accuracy.

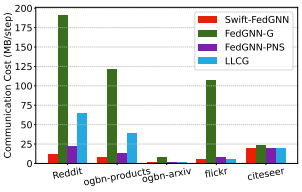


Figure 5: Average communication cost per training step.

Table 1: Total communication cost (GB) when achieving a target validation accuracy for each dataset.

	ogbn-products	Reddit	ogbn-arxiv	flickr	citeseer
Swift-FedGNN	**0.66**	**5.89**	**0.95**	**1.07**	**35.32**
FedGNN-G	8.40	70.72	7.91	17.22	43.43
LLCG	4.47	23.89	1.46	1.30	25.80
FedGNN-PNS	0.97	8.96	1.41	1.27	36.11

central server; 2) FedGNN-PNS [12]: An FGL framework in which each client periodically samples cross-client neighbors with an increasing sampling frequency and reuse the most recently sampled cross-client neighbors in the remaining iterations; and 3) FedGNN-G: A naive FGL algorithm that performs cross-client training on each client in every iteration.

3) Results: *3-1) Wall-clock Convergence Performance:* We first compare wall-clock convergence on two settings where communication and sampling bottlenecks are most severe: the large ogbn-products graph and the dense Reddit graph. Figures 4a and 4b show that for both datasets, Swift-FedGNN converges substantially faster than all baselines, which verifies its effectiveness in handling large or dense graphs. Despite less frequent cross-client training, Swift-FedGNN also attains validation accuracy comparable to FedGNN-G, which trains a GNN model without any information loss.

Although LLCG performs periodic cross-client training on the server, it uses the full neighbor set of the training nodes, leading to significant sampling and communication overhead. For instance, on ogbn-products, LLCG takes over 5000 ms to perform cross-client training, whereas Swift-FedGNN completes cross-client training within 200 ms due to neighbor sampling. Meanwhile, FedGNN-PNS gradually decreases the cross-client sampling interval during training, which leads to substantial sampling and communication overhead in later training stages and slows convergence.

3-2) Communication Cost: Figure 5 shows the average communication cost per training step across five datasets, demonstrating that Swift-FedGNN consistently incurs the lowest communication cost among all baselines. Specifically, compared with FedGNN-G, Swift-FedGNN reduces communication by $7\times$ to $21\times$ on Reddit, ogbn-products, ogbn-arxiv, and flickr. On the smallest graph, citeseer, the gap narrows because the size of cross-client neighbor information becomes negligible compared with the model size, yet Swift-FedGNN still maintains the lowest communication cost. For the largest dataset ogbn-products and the most dense dataset Reddit, Swift-FedGNN achieves $2\times$ and $5\times$ lower communication cost than FedGNN-PNS and LLCG, respectively. On the small datasets, ogbn-arxiv and flickr, the communication cost advantage of Swift-FedGNN remains evident, though closer to approximately $1\times$ lower than FedGNN-PNS and LLCG. These findings validate the superior communication efficiency of Swift-FedGNN.

Table 1 reports the total communication cost required to reach the same target validation accuracy on each dataset. The results demonstrate that Swift-FedGNN consistently incurs the lowest total communication cost across all datasets except citeseer. For example, to reach a target accuracy of 87% on ogbn-products, Swift-FedGNN achieves at least a 31.9% reduction in total communication cost compared to all baselines. Similarly, to reach a target accuracy of 55% on the smaller and sparser ogbn-arxiv dataset, Swift-FedGNN still delivers at least 32.2% communication savings, highlighting its strong communication efficiency across diverse graph structures.

3-3) Hyperparameter Sensitivity and Communication-Accuracy Trade-off: We explore the impact of two key parameters in Swift-FedGNN, the correction frequency I and the sampled-client budget K.

Table 2: Total communication cost (MB) of Swift-FedGNN under different choices of I and K on ogbn-products when achieving a target validation accuracy of 87%.

Sampled-Client Budget (K)	10				1	5	10	15
Correction Frequency (I)	5	10	20	40	10			
Communication Cost (MB)	1344.0	675.5	324.5	275.0	57.8	342.2	675.5	1027.4

Table 3: Validation accuracy of Swift-FedGNN under different choices of I and K on ogbn-products.

Sampled-Client Budget (K)	10				1	5	10	15
Correction Frequency (I)	5	10	20	40	10			
Validation Accuracy (%)	88.91	88.88	88.60	88.44	88.47	88.72	88.88	89.22

Table 4: Total communication cost (GB) on ogbn-products in two large-scale settings with 80 and 100 clients when achieving a target validation accuracy for each setting.

	Swift-FedGNN (Fixed $K = 10$)	Swift-FedGNN (Fixed $K/M = 1/2$)	FedGNN-G	LLCG	FedGNN-PNS
$M = 80$	0.69	2.42	37.26	3.90	4.49
$M = 100$	1.17	5.24	59.00	6.82	8.67

Table 5: Total communication cost (GB) on randomly partitioned ogbn-products when achieving a target validation accuracy of 89.5%.

	Swift-FedGNN	FedGNN-G	LLCG	FedGNN-PNS
Communication Cost (GB)	**1.44**	15.03	6.26	2.60

Table 6: Validation accuracy (%) on randomly partitioned ogbn-products.

	Swift-FedGNN	FedGNN-G	LLCG	FedGNN-PNS
Validation Accuracy (%)	**89.94**	91.23	89.92	89.91

Table 2 presents the total communication cost required to achieve a target accuracy of 87% on ogbn-products. The results show that increasing I or decreasing K substantially reduces communication cost. For example, increasing I from 5 to 40 saves approximately 80% of the communication overhead, while reducing K from 10 to 1 saves approximately 94%. Table 3 reports the corresponding validation accuracy of Swift-FedGNN. Increasing I from 5 to 40 leads to only a minor accuracy degradation (0.47%), and decreasing K from 15 to 1 also results in a minor accuracy drop (0.75%).

3-4) Evaluations in Extremely Large-Scale Settings: To evaluate scalability, we further evaluate Swift-FedGNN on ogbn-products under two extremely large-scale settings with 80 and 100 clients. Table 4 reports the total communication cost when achieving a target validation accuracy (*i.e.*, 83% for the 80-client setting and 84.3% for the 100-client setting). With fixed $K = 10$, Swift-FedGNN reduces total communication cost by at least 82% in the 80-client setting and 83% in the 100-client setting compared to all baselines. Even with a larger client sampling ratio of $K/M = 50\%$, Swift-FedGNN still achieves at least 38% and 23% communication savings in the 80-client and 100-client settings, respectively. These results demonstrate the effectiveness and scalability of Swift-FedGNN in large-scale FGL settings, and suggest that in such settings server-side aggregation remains manageable when K is chosen appropriately, *e.g.*, $K \ll M$.

3-5) Robustness under More Heterogeneous (Non-IID) Federated Settings: To evaluate the robustness of Swift-FedGNN under less structured scenarios, we replace METIS with random partitioning on ogbn-products. This introduces more heterogeneity by randomly assigning nodes to different subgraphs, thereby implicitly inducing non-identical and structurally unbalanced local subgraphs.

Table 5 shows that Swift-FedGNN reduces the total communication cost by at least 45% compared to all baselines to reach a target accuracy of 89.5%. Table 6 further demonstrates that Swift-FedGNN remains competitive in accuracy: among methods that do not rely on full graph training, *i.e.*, LLCG and FedGNN-PNS, Swift-FedGNN achieves the highest accuracy while using substantially less communication. Although FedGNN-G attains higher accuracy, it does so with far greater communication cost. Overall, these results confirm that Swift-FedGNN remains communication-efficient and accurate even under more heterogeneous random partitioning.

7 Conclusion

In this paper, we proposed Swift-FedGNN, which is a mini-batch-based and sampling-based federated graph learning framework, for efficient federated GNN training. Swift-FedGNN reduces the cross-client neighbor sampling and communication overhead by *periodically* sampling a subset of clients to conduct the GNN training on local graph data and cross-client neighbors, which is time-consuming. We theoretically proved that the convergence rate of Swift-FedGNN matches the SOTA rate of sampling-based GNN methods, even in more challenging federated settings. We conducted extensive numerical experiments on real-world graph datasets and verified the effectiveness of Swift-FedGNN.

References

[1] S. Deng, H. Rangwala, and Y. Ning, "Learning dynamic context graphs for predicting social events," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 1007–1016.

[2] J. Qiu, J. Tang, H. Ma, Y. Dong, K. Wang, and J. Tang, "Deepinf: Modeling influence locality in large social networks," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'18)*, 2018.

[3] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, "Graph convolutional neural networks for web-scale recommender systems," in *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2018, pp. 974–983.

[4] H. Wang, F. Zhang, M. Zhang, J. Leskovec, M. Zhao, W. Li, and Z. Wang, "Knowledge-aware graph neural networks with label smoothness regularization for recommender systems," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 968–977.

[5] Y. Wang, J. Wang, Z. Cao, and A. Barati Farimani, "Molecular contrastive learning of representations via graph neural networks," *Nature Machine Intelligence*, vol. 4, no. 3, pp. 279–287, 2022.

[6] K. Do, T. Tran, and S. Venkatesh, "Graph transformation policy network for chemical reaction prediction," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 750–760.

[7] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.

[8] B. Wang, A. Li, M. Pang, H. Li, and Y. Chen, "Graphfl: A federated learning framework for semi-supervised node classification on graphs," in *2022 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2022, pp. 498–507.

[9] J. Leskovec, L. A. Adamic, and B. A. Huberman, "The dynamics of viral marketing," *ACM Transactions on the Web (TWEB)*, vol. 1, no. 1, pp. 5–es, 2007.

[10] C. He, K. Balasubramanian, E. Ceyani, C. Yang, H. Xie, L. Sun, L. He, L. Yang, P. S. Yu, Y. Rong *et al.*, "Fedgraphnn: A federated learning system and benchmark for graph neural networks," in *ICLR 2021 Workshop on Distributed and Private Machine Learning (DPML)*, 2021.

[11] K. Zhang, C. Yang, X. Li, L. Sun, and S. M. Yiu, "Subgraph federated learning with missing neighbor generation," *Advances in Neural Information Processing Systems*, vol. 34, pp. 6671–6682, 2021.

[12] B. Du and C. Wu, "Federated graph learning with periodic neighbour sampling," in *2022 IEEE/ACM 30th International Symposium on Quality of Service (IWQoS)*. IEEE, 2022, pp. 1–10.

[13] Y. Yao, W. Jin, S. Ravi, and C. Joe-Wong, "Fedgcn: Convergence-communication tradeoffs in federated training of graph convolutional networks," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[14] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations*, 2017.

[15] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *Advances in neural information processing systems*, vol. 30, 2017.

[16] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *International Conference on Learning Representations*, 2019.

[17] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *International conference on machine learning*. PMLR, 2019, pp. 6861–6871.

[18] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations*, 2018.

[19] J. Chen, J. Zhu, and L. Song, "Stochastic training of graph convolutional networks with variance reduction," in *International Conference on Machine Learning*. PMLR, 2018, p. 942–950.

[20] J. Chen and R. Luss, "Stochastic gradient descent with biased but consistent gradient estimators," *arXiv preprint arXiv:1807.11880*, 2018.

[21] D. Zheng, C. Ma, M. Wang, J. Zhou, Q. Su, X. Song, Q. Gan, Z. Zhang, and G. Karypis, "Distdgl: Distributed graph neural network training for billion-scale graphs," in *2020 IEEE/ACM 10th Workshop on Irregular Applications: Architectures and Algorithms (IA3)*. IEEE, 2020, pp. 36–44.

[22] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[23] K. Zhao, W. Xiao, B. Ai, W. Shen, X. Zhang, Y. Li, and W. Lin, "AliGraph: An Industrial Graph Neural Network Platform," in *Proc. of SOSP Workshop on AI Systems*, 2019.

[24] J. Thorpe, Y. Qiao, J. Eyolfson, S. Teng, G. Hu, Z. Jia, J. Wei, K. Vora, R. Netravali, M. Kim *et al.*, "Dorylus: Affordable, scalable, and accurate gnn training with distributed cpu servers and serverless threads," in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, 2021, pp. 495–514.

[25] S. Gandhi and A. P. Iyer, "P3: Distributed deep graph learning at scale," in *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*, 2021, pp. 551–568.

[26] Z. Zhang, Z. Luo, and C. Wu, "Two-level graph caching for expediting distributed gnn training," in *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 2023, pp. 1–10.

[27] Z. Luo, Y. Bao, and C. Wu, "Optimizing task placement and online scheduling for distributed gnn training acceleration," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 890–899.

[28] C. Wan, Y. Li, C. R. Wolfe, A. Kyrillidis, N. S. Kim, and Y. Lin, "PipeGCN: Efficient full-graph training of graph convolutional networks with pipelined feature communication," in *International Conference on Learning Representations*, 2022.

[29] M. Ramezani, W. Cong, M. Mahdavi, M. Kandemir, and A. Sivasubramaniam, "Learn locally, correct globally: A distributed algorithm for training graph neural networks," in *International Conference on Learning Representations*, 2022.

[30] C. Wu, F. Wu, Y. Cao, Y. Huang, and X. Xie, "Fedgnn: Federated graph neural network for privacy-preserving recommendation," *International Workshop on Federated Learning for User Privacy and Data Confidentiality in Conjunction with ICML 2021*, 2021.

[31] V. Garg, S. Jegelka, and T. Jaakkola, "Generalization and representational limits of graph neural networks," in *International Conference on Machine Learning*. PMLR, 2020, pp. 3419–3430.

[32] W. Cong, M. Ramezani, and M. Mahdavi, "On the importance of sampling in training gcns: Tighter analysis and variance reduction," *arXiv preprint arXiv:2103.02696*, 2021.

[33] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open graph benchmark: Datasets for machine learning on graphs," *Advances in neural information processing systems*, vol. 33, pp. 22 118–22 133, 2020.

[34] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. Prasanna, "Graphsaint: Graph sampling based inductive learning method," in *International Conference on Learning Representations*, 2020.

[35] C. L. Giles, K. D. Bollacker, and S. Lawrence, "Citeseer: An automatic citation indexing system," in *Proceedings of the third ACM conference on Digital libraries*, 1998, pp. 89–98.

[36] G. Karypis and V. Kumar, "A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs," *SIAM Journal on Scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998.

[37] T. Chen, K. Zhou, K. Duan, W. Zheng, P. Wang, X. Hu, and Z. Wang, "Bag of tricks for training deeper graph neural networks: A comprehensive benchmark study," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 3, pp. 2769–2781, 2022.

[38] M. Wang, L. Yu, D. Zheng, Q. Gan, Y. Gai, Z. Ye, M. Li, J. Zhou, Q. Huang, C. Ma *et al.*, "Deep Graph Library: Towards Efficient and Scalable Deep Learning on Graphs," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[39] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.

[40] B. Yuan, Y. He, J. Davis, T. Zhang, T. Dao, B. Chen, P. S. Liang, C. Re, and C. Zhang, "Decentralized training of foundation models in heterogeneous environments," vol. 35, 2022, pp. 25 464–25 477.

A List of Notations

$\mathcal{G}(\mathcal{V}, \mathcal{E})$	Graph
$\mathcal{V}$	Set of nodes
$\mathcal{E}$	Set of edges
$N = \lvert\mathcal{V}\rvert$	Number of nodes
$\mathcal{M}$	Set of clients
$M = \lvert\mathcal{M}\rvert$	Number of clients
$\mathcal{G}^m(\mathcal{V}^m, \mathcal{E}^m)$	Subgraph at client m
$\mathcal{V}^m$	Set of nodes at client m
$\mathcal{E}^m$	Set of edges at client m
$\boldsymbol{x}_v^m \in \mathbb{R}^d$	Feature vector of node v at client m
y_v^m	Label of node v at client m
ℓ^m	Loss function (*e.g.*, cross-entropy loss) at client m
$\mathcal{V}_B^m$	Mini-batch of training nodes
$\boldsymbol{\theta} = \left\{\boldsymbol{W}^{(l)}\right\}_{l=1}^{L}$	Set of trainable model parameters
$m(v)$	Local client of node v
$\bar{m}(v)$	Remote client of node v
$\bar{\mathcal{M}}(v)$	Set of the remote clients that host the neighbors of node v
$\mathcal{N}^{m(v)}(v)$	Set of the neighbors of node v located on local client $m(v)$
$\mathcal{N}^{\bar{m}(v)}(v)$	Set of the neighbors of node v located on remote client $\bar{m}(v)$
$\boldsymbol{h}_v^{(l),m(v)}$	Embedding of node v located on client $m(v)$
$\boldsymbol{h}_{\mathcal{N}(v)}^{(l)}$	Aggregated embedding from node v's neighbors
$\boldsymbol{W}^{(l)}$	Weight matrix at l-th layer
$\sigma(\cdot)$	Activation function (*e.g.*, ReLU)
$\mathrm{AGG}(\cdot)$	Aggregation function (*e.g.*, mean)
$\mathrm{COMB}(\cdot)$	Combination function (*e.g.*, concatenation)
$\mathcal{B}_v^m$	Mini-batch of training nodes at client m
$\widetilde{\mathcal{S}} = \left\{\widetilde{\mathcal{S}}^{(l)}\right\}_{l=0}^{L-1}$	Subset of L-hop neighbors for the training nodes in $\mathcal{B}_v^m$
$\widetilde{\mathcal{N}}^m(v)$	Set of the sampled neighbors located on client m for node v
$\mathcal{K}$	Set of sampled clients for cross-client training
$K = \lvert\mathcal{K}\rvert$	Number of sampled clients for cross-client training
$\widetilde{\mathcal{M}}(v)$	Set of remote clients that host the sampled cross-client neighbors of the training node v
$\widetilde{m}(v)$	Remote client with respect to $m(v)$
$\nabla \widetilde{F}^m(\boldsymbol{\theta}_t^m)$	Stochastic gradient
α	Learning rate
$\boldsymbol{A} \in \mathbb{R}^{N \times N}$	Adjacency matrix of graph $\mathcal{G}$
$\boldsymbol{P}$	Propagation matrix
$\boldsymbol{D}$	Degree matrix
$\boldsymbol{A}^m$	Adjacency matrix of subgraph $\mathcal{G}^m$
$\boldsymbol{A}_{local}^m$	Adjacency matrix corresponds to the nodes located on client m

$\boldsymbol{A}^m_{remote}$	Adjacency matrix corresponds to the cross-client neighbors located on the remote clients other than m
$\boldsymbol{D}^m$	Degree matrix of client m
$\boldsymbol{P}^m$	Propagation matrix of client m
$\boldsymbol{P}^m_{local}$	Propagation matrix corresponds to the nodes located on client m
$\boldsymbol{P}^m_{remote}$	Propagation matrix corresponds to the cross-client neighbors located on the remote clients other than m

B Additional Experimental Details and Results

This appendix provides additional experimental details and additional experimental results, including further evidence for the theoretical findings, evaluations on another widely used GNN model, GIN [16], and additional validation-accuracy comparisons.

B.1 Additional Experimental Results

B.1.1 Experimental Support for Theoretical Findings

Table 7: Gradient bias under varying GNN depths on ogbn-arxiv.

GNN Depth	2	14	16
Gradient Bias	0.46	17.56	30.11

Table 8: Validation accuracy (%) under varying GNN depths on ogbn-arxiv.

GNN Depth	2	14	16
Validation Accuracy (%)	57.17	54.60	48.46

To empirically support our theoretical findings, we use the bias between the full and stochastic gradients as an empirical proxy, which has a theoretical upper bound of $LB^l_{\Delta G}$ (see Lemma 5.4) Table 7 presents the gradient bias on ogbn-arxiv across different GNN depths. The results show that the gradient bias increases with the GNN depth, consistent with our theoretical finding that deeper GNNs incur larger bias due to amplified sampling and cross-client neighbor errors.

Table 8 reports validation accuracy and shows that the validation accuracy degrades as GNN depth increases. This trend aligns with Theorem 5.6 that deeper GNNs introduce larger gradient bias, which in turn increases approximation error and hurts performance. Overall, these results provide empirical support for the validity and practical relevance of our theoretical error bounds.

We emphasize that this depth-related degradation is a fundamental issue of GNN architectures rather than a limitation of Swift-FedGNN itself. It is widely observed across GNN methods, and can often be alleviated through architecture-level improvements, many of which, such as [37], are compatible with Swift-FedGNN.

B.1.2 Evaluation Across Different GNN Models

Table 9: Performance comparison using the GIN model when achieving a target validation accuracy for each dataset.

	ogbn-products		ogbn-arxiv	
	Total Comm. Cost (GB)	Running Time (s)	Total Comm. Cost (GB)	Running Time (s)
Swift-FedGNN	**0.74**	**65.18**	**2.29**	**75.80**
LLCG	3.74	223.77	3.81	103.98
FedGNN-PNS	5.75	113.46	3.62	131.92
FedGNN-G	38.12	767.53	19.36	575.26

To assess the adaptivity of Swift-FedGNN across GNN architectures, we further evaluate it with GIN [16] on multiple datasets. Table 9 reports the total communication cost and wall-clock time

Table 10: Validation accuracy (%) using GIN.

	Swift-FedGNN	LLCG	FedGNN-PNS	FedGNN-G
ogbn-products	**81.93**	80.72	78.70	83.76
ogbn-arxiv	**56.69**	57.32	56.54	57.01

when reaching a target validation accuracy of 80% on ogbn-products and 56% on ogbn-arxiv. In both cases, Swift-FedGNN achieves the lowest wall-clock time and reduces the total communication cost by at least 80% on ogbn-products and 37% on ogbn-arxiv compared to all baselines.

Table 10 further shows that, similar to the GraphSAGE model, Swift-FedGNN attains validation accuracy comparable to the baselines while significantly reducing sampling and communication overhead. These results show that the communication-efficiency benefits of Swift-FedGNN are not specific to GraphSAGE, but extend to other element-wise message-passing GNNs such as GIN without sacrificing accuracy.

B.1.3 Validation Accuracy Comparisons

Table 11: Validation accuracy (%) using GraphSAGE.

	ogbn-products	Reddit	ogbn-arxiv	flickr	citeseer
Swift-FedGNN	**88.88**	**95.47**	**57.17**	**50.19**	**66.00**
LLCG	87.66	95.27	56.78	50.12	68.40
FedGNN-PNS	87.89	95.46	55.86	51.47	66.27
FedSage	88.15	95.30	56.55	49.75	64.39
FedGNN-G	88.71	95.96	56.78	51.57	66.08

Evaluations with the GraphSAGE Model. Table 11 shows the validation accuracy of GraphSAGE. To assess the impact of cross-client neighbors, we additionally include FedSage [11], an FGL algorithm that ignores them entirely and performs purely local training. Despite much lower sampling and communication overhead, Swift-FedGNN achieves validation accuracy comparable to the more communication-intensive baselines and consistently outperforms FedSage across all datasets, highlighting the importance of incorporating cross-client neighbor information. In particular, Swift-FedGNN attains the best accuracy on ogbn-products and ogbn-arxiv. Although some baselines achieve slightly higher accuracy on Reddit, flickr, and citeseer, they do so with substantially more communication. This is partly because these smaller datasets suffer greater information loss after partitioning, so frequent cross-client exchange can be more beneficial. However, such datasets are less representative of the large-scale geo-distributed settings that motivate FGL. On the larger ogbn-products dataset, which better matches this regime, Swift-FedGNN achieves the best validation accuracy while remaining substantially more communication-efficient.

Table 12: Validation accuracy (%) on ogbn-products in two large-scale settings with 80 and 100 clients.

	Swift-FedGNN ($K = 10$)	Swift-FedGNN ($K/M = 1/2$)	LLCG	FedGNN-PNS	FedSage	FedGNN-G
$M = 80$	85.74	86.12	83.54	85.67	85.35	86.52
$M = 100$	85.07	85.53	84.35	85.15	84.41	85.63

Evaluations in Extremely Large-Scale Settings. Table 12 presents the validation accuracy on ogbn-products in two extremely large-scale settings with 80 and 100 clients. The results show that Swift-FedGNN maintains comparable or better validation accuracy in both settings, with larger K yielding slightly improved performance. Specifically, with a fixed sampled-client budget of $K = 10$, Swift-FedGNN remains close in accuracy to the more communication-intensive baselines. Increasing the client sampling budget to $K/M = 1/2$ yields a small additional improvement, consistent with Theorem 5.6: more sampled-client budget improves accuracy but increases communication cost. Overall, these results show that Swift-FedGNN scales predictably with the number of clients, and that even small sampled-client budget is sufficient to maintain strong accuracy while preserving communication savings.

Table 13: Benchmark datasets and key parameters.

Dataset	# of Nodes	# of Edges	Edges per Node
ogbn-products	2.4 M	61.9 M	25.8
Reddit	233 K	114.6 M	491.4
ogbn-arxiv	169 K	1.2 M	7.1
flickr	89 K	900 K	10.1
citeseer	3327	9228	2.8

B.2 Additional Experimental Details

Dataset. Table 13 summarizes the key statistics of the datasets used in our experiments, including: 1) ogbn-products [33], which is an Amazon product co-purchasing graph derived from [9]; 2) Reddit [15], which consists of online forum posts within a month, where posts commented on by the same user are connected by an edge; 3) ogbn-arxiv [33], which is a citation network between arXiv papers in the field of computer science, where nodes represent papers and directed edges indicate citation links; 4) flickr [34], which is an image network where each node represents an image and edges connect images that share common properties such as tags or visual similarity; and 5) citeseer [35], which is a citation graph of research papers, where each node denotes a document and edges represent citation relationships between them.

Implementation and Testbed. We implement Swift-FedGNN using Python on DGL 2.0.0 [38] and PyTorch 2.2.1 [39]. Our implementation includes a custom GPU-based sampler built on top of DGL's native sampler, which is designed to sequentially sample local and remote neighbors for each client at every layer. Additionally, we customized the GraphSAGE layer and GIN layer to facilitate model-parallel training within Swift-FedGNN . In this setup, the server handles the sampling and aggregation of node features and intermediate activations, while the clients are responsible for executing the nonlinear computations associated with the GraphSAGE layer.

We simulate a real-world federated learning scenario using a single machine equipped with NVIDIA Tesla V100 GPUs and 64GB memory. In our setup, both the clients and the server operate on the GPU, and data communication between them is simulated using shared memory. We monitor the data transfer size between the server and clients and set a simulated cross-client network bandwidth at 1 Gbps, aligning with real-world measurements reported in [40].

GNN Models. We train a two-layer GraphSAGE model and a two-layer GIN model with a hidden dimension of 256. Uniform sampling is employed for neighbor sampling, with fan-outs—*i.e.*, the number of sampled neighbors—set according to the official training script provided by the DGL team. The fan-out values are set to $[20, 15]$ for the ogbn-products dataset, and $[15, 10]$ for all other datasets.

Hyperparameters. The training mini-batch size is set at 256. For optimization, we use the Adam optimizer with a weight decay of 5×10^{-4}. We use a learning rate of 0.01 for the ogbn-products dataset, 0.0001 for the flickr dataset, 0.00001 for the citeseer dataset, and 0.001 for both the ogbn-arxiv and Reddit datasets. In Swift-FedGNN, we set $K = 10$ for the ogbn-products dataset and $K = 5$ for all other datasets. We choose $I = 5$ for the citeseer dataset and $I = 10$ for the remaining datasets.

C Proof Sketch of Theorem 5.6

In this appendix, we present a proof sketch of Theorem 5.6 and structure the proof as follows.

Step 1: Decompose the local and cross-client gradients layer-wise. For an L-layer GCN, when client m performs local training, each sampling-based GCN layer executes one feature propagation step, defined as:

$$\widetilde{\boldsymbol{H}}_{local}^{(l),m} \triangleq \sigma\left(\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}\boldsymbol{W}^{(l),m}\right).$$

Using the chain rule, the stochastic gradient can be computed as $\nabla\widetilde{F}^m\left(\boldsymbol{\theta}^m\right) = \left\{\widetilde{\boldsymbol{G}}_{local}^{(l),m}\right\}_{l=1}^{L}$, where

$$\widetilde{\boldsymbol{G}}_{local}^{(l),m} \triangleq \left[\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}\right]^{\top}\widetilde{\boldsymbol{D}}_{local}^{(l),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{local}^{(l),m}\right)\Bigg],$$

$$\widetilde{\boldsymbol{D}}_{local}^{(l),m} \triangleq \left[\widetilde{\boldsymbol{P}}_{local}^{(l+1),m}\right]^\top \widetilde{\boldsymbol{D}}_{local}^{(l+1),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{local}^{(l+1),m}\right)\left[\boldsymbol{W}^{(l+1),m}\right]^\top\Bigg],$$

in which $\circ$ is Hadamard product, $\widetilde{\boldsymbol{Z}}_{local}^{(l),m} = \widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}\boldsymbol{W}^{(l),m}$, and $\widetilde{\boldsymbol{D}}_{local}^{(L),m} = \partial\ell^m\left(\widetilde{\boldsymbol{H}}_{local}^{(L),m}, \boldsymbol{Y}_{local}^m\right)/\partial\widetilde{\boldsymbol{H}}_{local}^{(L),m}$.

Similarly, when client m conducts cross-client training, each sampling-based GCN layer can be defined as:

$$\widetilde{\boldsymbol{H}}_{full}^{(l),m} \triangleq \sigma\left(\left(\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m} + \widetilde{\boldsymbol{P}}_{remote}^{(l),m}\widetilde{\boldsymbol{H}}_{remote}^{(l-1),m}\right)\boldsymbol{W}^{(l),m}\right).$$

Using the chain rule, the stochastic gradient can be calculated as $\nabla\widetilde{F}^m(\boldsymbol{\theta}^m) = \left\{\widetilde{\boldsymbol{G}}_{full}^{(l),m}\right\}_{l=1}^L$, where

$$\widetilde{\boldsymbol{G}}_{full}^{(l),m} \triangleq \left[\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m} + \widetilde{\boldsymbol{P}}_{remote}^{(l),m}\widetilde{\boldsymbol{H}}_{remote}^{(l-1),m}\right]^\top \widetilde{\boldsymbol{D}}_{full}^{(l),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l),m}\right),$$
$$\widetilde{\boldsymbol{D}}_{full}^{(l),m} \triangleq \left[\widetilde{\boldsymbol{P}}_{local}^{(l+1),m} + \widetilde{\boldsymbol{P}}_{remote}^{(l+1),m}\right]^\top \widetilde{\boldsymbol{D}}_{full}^{(l+1),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l+1),m}\right)\left[\boldsymbol{W}^{(l+1),m}\right]^\top,$$

in which $\widetilde{\boldsymbol{Z}}_{full}^{(l),m} = \left(\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m} + \widetilde{\boldsymbol{P}}_{remote}^{(l),m}\widetilde{\boldsymbol{H}}_{remote}^{(l-1),m}\right)\boldsymbol{W}^{(l),m}$, and $\widetilde{\boldsymbol{D}}_{full}^{(L),m} = \partial\ell^m\left(\widetilde{\boldsymbol{H}}_{full}^{(L),m}, \boldsymbol{Y}_{full}^m\right)/\partial\widetilde{\boldsymbol{H}}_{full}^{(L),m}$.

Step 2: Establish bounds on the embeddings and backward signals. Under Assumptions 5.1–5.3, and for any $l \in [L]$, the Frobenius norm of node embedding matrices, gradient passing from the l-th layer node embeddings to the $(l-1)$-th are bounded, *i.e.*,

$$\left\|\boldsymbol{H}_{local}^{(l),m}\right\|_F, \left\|\widetilde{\boldsymbol{H}}_{local}^{(l),m}\right\|_F \le B_H^l, \quad \left\|\boldsymbol{H}_{full}^{(l),m}\right\|_F, \left\|\widetilde{\boldsymbol{H}}_{full}^{(l),m}\right\|_F \le B_H^f,$$
$$\left\|\boldsymbol{D}_{local}^{(l),m}\right\|_F, \left\|\widetilde{\boldsymbol{D}}_{local}^{(l),m}\right\|_F \le B_D^l, \quad \left\|\boldsymbol{D}_{full}^{(l),m}\right\|_F, \left\|\widetilde{\boldsymbol{D}}_{full}^{(l),m}\right\|_F \le B_D^f,$$

where

$$B_H^l, B_H^f = \max_{1\le l\le L}\left(C_\sigma B_P B_W\right)^l B_X, \quad B_D^l, B_D^f = \max_{1\le l\le L}\left(B_P B_W C_\sigma\right)^{L-l} C_l.$$

Step 3: Bound the sampling-induced errors. Under Assumptions 5.1–5.3, and for any $l \in [L]$, the errors caused by sampling are bounded, *i.e.*,

$$\left\|\widetilde{\boldsymbol{H}}_{local}^{(l),m} - \boldsymbol{H}_{local}^{(l),m}\right\|_F \le B_{\Delta H}^l, \quad \left\|\widetilde{\boldsymbol{H}}_{full}^{(l),m} - \boldsymbol{H}_{full}^{(l),m}\right\|_F \le B_{\Delta H}^f,$$
$$\left\|\widetilde{\boldsymbol{D}}_{local}^{(l),m} - \boldsymbol{D}_{local}^{(l),m}\right\|_F \le B_{\Delta D}^l, \quad \left\|\widetilde{\boldsymbol{D}}_{full}^{(l),m} - \boldsymbol{D}_{full}^{(l),m}\right\|_F \le B_{\Delta D}^f,$$

where

$$B_{\Delta H}^l = \max_{1\le l\le L}\left(\left(C_\sigma B_W B_H^l B_{\Delta P}^l\right)^l + \left(C_\sigma B_W B_P\right)^l B_{\Delta X}^l\right),$$
$$B_{\Delta H}^f = \max_{1\le l\le L}\left(\left(C_\sigma B_W B_H^f B_{\Delta P}^f\right)^l + \left(C_\sigma B_W B_P\right)^l B_{\Delta X}^f\right),$$
$$B_{\Delta D}^l = \max_{1\le l\le L}\Bigg(\Big(B_W B_D^l C_\sigma B_{\Delta P}^l + B_W^2 B_P B_D^l L_\sigma B_H^l B_{\Delta P}^l + B_W^2 B_P^2 B_D^l L_\sigma B_{\Delta H}^l\Big)^{L-l} + \left(B_W B_P C_\sigma\right)^{L-l} L_l B_{\Delta H}^l\Bigg),$$
$$B_{\Delta D}^f = \max_{1\le l\le L}\Bigg(\Big(B_W B_D^f C_\sigma B_{\Delta P}^f + B_W^2 B_P B_D^f L_\sigma B_H^f B_{\Delta P}^f + B_W^2 B_P^2 B_D^f L_\sigma B_{\Delta H}^f\Big)^{L-l} + \left(B_W B_P C_\sigma\right)^{L-l} L_l B_{\Delta H}^f\Bigg).$$

Step 4: Bound the errors due to missing cross-client neighbors. Under Assumptions 5.1–5.3, and for any $l \in [L]$, the errors caused by the information loss of the cross-client neighbors are bounded, *i.e.*,

$$\left\| \boldsymbol{H}_{local}^{(l),m} - \boldsymbol{H}_{full}^{(l),m} \right\|_F \leq B_{\Delta H}^r, \quad \left\| \boldsymbol{D}_{local}^{(l),m} - \boldsymbol{D}_{full}^{(l),m} \right\|_F \leq B_{\Delta D}^r,$$

where

$$\begin{aligned} B_{\Delta H}^r &= \max_{1 \leq l \leq L} \left(\left(C_\sigma B_W B_P \right)^l B_X^r + \left(C_\sigma B_W B_H^f B_P \right)^l \right), \\ B_{\Delta D}^r &= \max_{1 \leq l \leq L} \Bigg(\Big(B_W B_D^l C_\sigma B_P + B_W^2 B_P^2 B_D^f L_\sigma B_H^l \\ &\qquad + B_W^2 B_P^2 B_D^f L_\sigma B_{\Delta H}^r \Big)^{L-l} + \left(B_W B_P C \sigma \right)^{L-l} L_l B_{\Delta H}^r \Bigg). \end{aligned}$$

Step 5: Convert representation errors into gradient errors. Under Assumptions 5.1–5.3, the errors between the stochastic gradients and the full gradients, and the full gradients computed with and without the cross-client neighbors are bounded as follows:

$$\begin{aligned} \left\| \nabla F_{local}^m \left(\boldsymbol{\theta}^m \right) - \nabla \widetilde{F}_{local}^m \left(\boldsymbol{\theta}^m \right) \right\|_F &\leq L B_{\Delta G}^l, \\ \left\| \nabla F_{full}^m \left(\boldsymbol{\theta}^m \right) - \nabla \widetilde{F}_{full}^m \left(\boldsymbol{\theta}^m \right) \right\|_F &\leq L B_{\Delta G}^f, \\ \left\| \nabla F_{full}^m \left(\boldsymbol{\theta}^m \right) - \nabla F_{local}^m \left(\boldsymbol{\theta}^m \right) \right\|_F &\leq L B_{\Delta G}^r, \end{aligned}$$

where

$$\begin{aligned} B_{\Delta G}^l = \max_{1 \leq l \leq L} \Bigg(& \left(B_D^l C_\sigma + B_P B_H^l B_D^l L_\sigma B_W \right) B_H^l B_{\Delta P}^l + B_P B_H^l C_\sigma B_{\Delta D}^l \\ & + \left(B_D^l C_\sigma + B_P B_H^l B_D^l L_\sigma B_W \right) B_P B_{\Delta H}^l \Bigg), \end{aligned} \tag{9}$$

$$\begin{aligned} B_{\Delta G}^f = \max_{1 \leq l \leq L} \Bigg(& \left(B_D^f C_\sigma + B_P B_H^f B_D^f L_\sigma B_W \right) B_H^f B_{\Delta P}^f + B_P B_H^f C_\sigma B_{\Delta D}^f \\ & + \left(B_D^f C_\sigma + B_P B_H^f B_D^f L_\sigma B_W \right) B_P B_{\Delta H}^f \Bigg), \end{aligned} \tag{10}$$

$$\begin{aligned} B_{\Delta G}^r = \max_{1 \leq l \leq L} \Bigg(& \left(B_D^l C_\sigma + B_P B_H^f B_D^f L_\sigma B_W \right) B_P B_{\Delta H}^r + B_P B_H^f C_\sigma B_{\Delta D}^r \\ & + \left(B_D^l C_\sigma + B_P B_H^f B_D^f L_\sigma B_W \right) B_H^f B_P \Bigg). \end{aligned} \tag{11}$$

Step 6: Prove a one-step descent inequality. By L_F-smoothness of the global loss,

$$\begin{aligned} & \mathcal{L} \left(\boldsymbol{\theta}_{t+1} \right) - \mathcal{L} \left(\boldsymbol{\theta}_t \right) \\ & \leq -\frac{\alpha}{2} \left\| \nabla \mathcal{L} \left(\boldsymbol{\theta}_t \right) \right\|^2 + \frac{\alpha}{2} \frac{1}{M} \sum_{m \in \mathcal{M}} \left\| \nabla F^m \left(\boldsymbol{\theta}_t^m \right) - \nabla \widetilde{F}^m \left(\boldsymbol{\theta}_t^m \right) \right\|^2. \end{aligned}$$

Step 7: Bound the gradient errors according to the training mode. When $t \in [(n_t - 1) I + 1, n_t I - 1] \cap \mathbb{Z}$, where $n_t = \{1, 2, \cdots\}$, all clients $m \in \mathcal{M}$ conducts local training. Thus,

$$\begin{aligned} \left\| \nabla F^m \left(\boldsymbol{\theta}_t^m \right) - \nabla \widetilde{F}^m \left(\boldsymbol{\theta}_t^m \right) \right\| &= \left\| \nabla F_{full}^m \left(\boldsymbol{\theta}_t^m \right) - \nabla \widetilde{F}_{local}^m \left(\boldsymbol{\theta}_t^m \right) \right\| \\ &\leq L B_{\Delta G}^r + L B_{\Delta G}^l. \end{aligned}$$

When $t = n_t I$, where $n_t = \{1, 2, \cdots\}$, the randomly sampled clients $m \in \mathcal{K}$ conduct cross-client training, and thus

$$\left\| \nabla F^m\left(\boldsymbol{\theta}_t^m\right) - \nabla \widetilde{F}^m\left(\boldsymbol{\theta}_t^m\right) \right\| = \left\| \nabla F_{full}^m\left(\boldsymbol{\theta}_t^m\right) - \nabla \widetilde{F}_{full}^m\left(\boldsymbol{\theta}_t^m\right) \right\| \leq L B_{\Delta G}^f.$$

Step 8: Telescope over all training rounds.

Summing the one-step descent inequality from $i = (n_t - 1) I + 1$ to $n_t I$ and choosing $T = n_t I$ yields

$$\begin{aligned}\sum_{t=0}^{T-1} \left(\mathcal{L}\left(\boldsymbol{\theta}_{t+1}\right) - \mathcal{L}\left(\boldsymbol{\theta}_t\right)\right) \leq & -\frac{\alpha}{2} \sum_{t=0}^{T-1} \left\| \nabla \mathcal{L}\left(\boldsymbol{\theta}_t\right) \right\|^2 + n_t \frac{\alpha}{2M} K L^2 \left(B_{\Delta G}^f\right)^2 \\ & + \frac{\alpha}{2}(T - n_t) L^2 \left(B_{\Delta G}^l + B_{\Delta G}^r\right)^2 + n_t \frac{\alpha}{2M}(M - K) L^2 \left(B_{\Delta G}^l + B_{\Delta G}^r\right)^2.\end{aligned}$$

Step 9: Choose the step size and conclude. Rearranging the terms, multiplying both sides by $2/\alpha$, dividing both sides by T, and choosing $\alpha = \sqrt{M}/\sqrt{T}$ yields the desired result and completes the proof of Theorem 5.6.

D Proof of Theorem 5.6

In this appendix, we present a detailed proof of Theorem 5.6.

D.1 Layer-Wise Gradient Computations in Swift-FedGNN

Recall that Swift-FedGNN uses GCN [14] as the architecture of GNN to prove the convergence performance. When client m performs local training that updates the local GNN model using only the local graph data, Each sampling-based GCN layer executes one feature propagation step, defined as:

$$\widetilde{\boldsymbol{H}}_{local}^{(l),m} = \left[\widetilde{f}^{(l),m}\left(\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}, \boldsymbol{W}^{(l),m}\right) \triangleq \sigma\left(\widetilde{\boldsymbol{P}}_{local}^{(l),m} \widetilde{\boldsymbol{H}}_{local}^{(l-1),m} \boldsymbol{W}^{(l),m}\right)\right].$$

Using the chain rule, the stochastic gradient can be computed as $\nabla \widetilde{F}^m\left(\boldsymbol{\theta}^m\right) = \left\{\widetilde{\boldsymbol{G}}_{local}^{(l),m}\right\}_{l=1}^L$, where

$$\begin{aligned}\widetilde{\boldsymbol{G}}_{local}^{(l),m} = & \Bigg[\nabla_W \widetilde{f}^{(l),m}\left(\widetilde{\boldsymbol{D}}_{local}^{(l),m}, \widetilde{\boldsymbol{H}}_{local}^{(l-1),m}, \boldsymbol{W}^{(l),m}\right) \\ & \triangleq \left[\widetilde{\boldsymbol{P}}_{local}^{(l),m} \widetilde{\boldsymbol{H}}_{local}^{(l-1),m}\right]^\top \widetilde{\boldsymbol{D}}_{local}^{(l),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{local}^{(l),m}\right)\Bigg], \\ \widetilde{\boldsymbol{D}}_{local}^{(l),m} = & \Bigg[\nabla_H \widetilde{f}^{(l+1),m}\left(\widetilde{\boldsymbol{D}}_{local}^{(l+1),m}, \widetilde{\boldsymbol{H}}_{local}^{(l),m}, \boldsymbol{W}^{(l+1),m}\right) \\ & \triangleq \left[\widetilde{\boldsymbol{P}}_{local}^{(l+1),m}\right]^\top \widetilde{\boldsymbol{D}}_{local}^{(l+1),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{local}^{(l+1),m}\right)\left[\boldsymbol{W}^{(l+1),m}\right]^\top\Bigg],\end{aligned}$$

in which $\widetilde{\boldsymbol{Z}}_{local}^{(l),m} = \widetilde{\boldsymbol{P}}_{local}^{(l),m} \widetilde{\boldsymbol{H}}_{local}^{(l-1),m} \boldsymbol{W}^{(l),m}$, $\widetilde{\boldsymbol{D}}_{local}^{(L),m} = \partial \ell^m\left(\widetilde{\boldsymbol{H}}_{local}^{(L),m}, \boldsymbol{Y}_{local}^m\right) / \partial \widetilde{\boldsymbol{H}}_{local}^{(L),m}$, and $\circ$ represents Hadamard product.

Similarly, when client m conducts cross-client training that updates the local GNN model based on the local graph data and the cross-client neighbors, each sampling-based GNN layer can be defined as:

$$\widetilde{\boldsymbol{H}}_{full}^{(l),m} = \left[\widetilde{f}^{(l),m}\left(\widetilde{\boldsymbol{H}}_{full}^{(l-1),m}, \boldsymbol{W}^{(l),m}\right) \triangleq \sigma\left(\left(\widetilde{\boldsymbol{P}}_{local}^{(l),m} \widetilde{\boldsymbol{H}}_{local}^{(l-1),m} + \widetilde{\boldsymbol{P}}_{remote}^{(l),m} \widetilde{\boldsymbol{H}}_{remote}^{(l-1),m}\right) \boldsymbol{W}^{(l),m}\right)\right].$$

Using the chain rule, the stochastic gradient can be calculated as $\nabla \widetilde{F}^m\left(\boldsymbol{\theta}^m\right) = \left\{\widetilde{\boldsymbol{G}}_{full}^{(l),m}\right\}_{l=1}^L$, where

$$\widetilde{\boldsymbol{G}}_{full}^{(l),m} = \Bigg[\nabla_W \widetilde{f}^{(l),m}\left(\widetilde{\boldsymbol{D}}_{full}^{(l),m}, \widetilde{\boldsymbol{H}}_{full}^{(l-1),m}, \boldsymbol{W}^{(l),m}\right)$$

$$\triangleq \left[\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}+\widetilde{\boldsymbol{P}}_{remote}^{(l),m}\widetilde{\boldsymbol{H}}_{remote}^{(l-1),m}\right]^{\top}\widetilde{\boldsymbol{D}}_{full}^{(l),m}\circ\nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l),m}\right)\Bigg],$$

$$\widetilde{\boldsymbol{D}}_{full}^{(l),m}=\Bigg[\nabla_H\widetilde{f}^{(l+1),m}\left(\widetilde{\boldsymbol{D}}_{full}^{(l+1),m},\widetilde{\boldsymbol{H}}_{full}^{(l),m},\boldsymbol{W}^{(l+1),m}\right)$$

$$\triangleq \left[\widetilde{\boldsymbol{P}}_{local}^{(l+1),m}+\widetilde{\boldsymbol{P}}_{remote}^{(l+1),m}\right]^{\top}\widetilde{\boldsymbol{D}}_{full}^{(l+1),m}\circ\nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l+1),m}\right)\left[\boldsymbol{W}^{(l+1),m}\right]^{\top}\Bigg],$$

in which $\widetilde{\boldsymbol{Z}}_{full}^{(l),m}=\left(\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}+\widetilde{\boldsymbol{P}}_{remote}^{(l),m}\widetilde{\boldsymbol{H}}_{remote}^{(l-1),m}\right)\boldsymbol{W}^{(l),m}$, and $\widetilde{\boldsymbol{D}}_{full}^{(L),m}=\partial\ell^m\left(\widetilde{\boldsymbol{H}}_{full}^{(L),m},\boldsymbol{Y}_{full}^m\right)/\partial\widetilde{\boldsymbol{H}}_{full}^{(L),m}$.

D.2 Useful Propositions and Lemmas

Proposition D.1. *Under Assumption 5.3, the inequalities in Table 14 and Table 15 are hold.*

Table 14: Upper-bound for the norms of the propagation matrix and the node feature matrix.

	PROPAGATION MATRIX	NODE FEATURE MATRIX
FULL GRAPH	$\left\|\boldsymbol{P}_{full}\right\|_F\leq B_P$	$\left\|\boldsymbol{X}_{full}\right\|_F\leq B_X$
LOCAL GRAPH	$\left\|\boldsymbol{P}_{local}\right\|_F\leq B_P^l\leq B_P$	$\left\|\boldsymbol{X}_{local}\right\|_F\leq B_X^l\leq B_X$
CROSS-CLIENT NEIGHBORS	$\left\|\boldsymbol{P}_{remote}\right\|_F\leq B_P^r\leq B_P$	$\left\|\boldsymbol{X}_{remote}\right\|_F\leq B_X^r\leq B_X$

Table 15: Relationships for the norms of the propagation matrix and the node feature matrix before and after sampling.

	PROPAGATION MATRIX	NODE FEATURE MATRIX
FULL GRAPH	$\left\|\widetilde{\boldsymbol{P}}_{full}-\boldsymbol{P}_{full}\right\|_F\leq B_{\Delta P}^f$	$\left\|\widetilde{\boldsymbol{X}}_{full}-\boldsymbol{X}_{full}\right\|_F\leq B_{\Delta X}^f$
LOCAL GRAPH	$\left\|\widetilde{\boldsymbol{P}}_{local}-\boldsymbol{P}_{local}\right\|_F\leq B_{\Delta P}^l$	$\left\|\widetilde{\boldsymbol{X}}_{local}-\boldsymbol{X}_{local}\right\|_F\leq B_{\Delta X}^l$
CROSS-CLIENT NEIGHBORS	$\left\|\widetilde{\boldsymbol{P}}_{remote}-\boldsymbol{P}_{remote}\right\|_F\leq B_{\Delta P}^r$	$\left\|\widetilde{\boldsymbol{X}}_{remote}-\boldsymbol{X}_{remote}\right\|_F\leq B_{\Delta X}^r$

Lemma D.2. *[Lemma 1 in [32]] An L-later GCN is L_F-Lipschitz smooth, i.e., $\left\|\nabla\mathcal{L}\left(\boldsymbol{\theta}_1\right)-\nabla\mathcal{L}\left(\boldsymbol{\theta}_2\right)\right\|_F\leq L_F\left\|\boldsymbol{\theta}_1-\boldsymbol{\theta}_2\right\|_F$.*

D.3 Bounds on Embeddings and Backward Signals

Lemma D.3. *Under Assumptions 5.1–5.3, and for any $l\in[L]$, the Frobenius norm of node embedding matrices, gradient passing from the l-th layer node embeddings to the $(l-1)$-th are bounded, i.e.,*

$$\left\|\boldsymbol{H}_{local}^{(l),m}\right\|_F,\left\|\widetilde{\boldsymbol{H}}_{local}^{(l),m}\right\|_F\leq B_H^l,\qquad \left\|\boldsymbol{H}_{full}^{(l),m}\right\|_F,\left\|\widetilde{\boldsymbol{H}}_{full}^{(l),m}\right\|_F\leq B_H^f,$$
$$\left\|\boldsymbol{D}_{local}^{(l),m}\right\|_F,\left\|\widetilde{\boldsymbol{D}}_{local}^{(l),m}\right\|_F\leq B_D^l,\qquad \left\|\boldsymbol{D}_{full}^{(l),m}\right\|_F,\left\|\widetilde{\boldsymbol{D}}_{full}^{(l),m}\right\|_F\leq B_D^f,$$

where

$$B_H^l,B_H^f=\max_{1\leq l\leq L}\left(C_\sigma B_P B_W\right)^l B_X,\qquad B_D^l,B_D^f=\max_{1\leq l\leq L}\left(B_P B_W C_\sigma\right)^{L-l}C_l.$$

Proof.

$$\begin{aligned}\left\|\boldsymbol{H}_{local}^{(l),m}\right\|_F&=\left\|\sigma\left(\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}\boldsymbol{W}^{(l),m}\right)\right\|_F\\&\overset{(a)}{\leq}C_\sigma B_W\left\|\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}\right\|_F\leq C_\sigma B_W\left\|\boldsymbol{P}_{local}^{(l),m}\right\|\left\|\boldsymbol{H}_{local}^{(l-1),m}\right\|_F\\&\overset{(b)}{\leq}C_\sigma B_W B_P\left\|\boldsymbol{H}_{local}^{(l-1),m}\right\|_F\leq\left(C_\sigma B_W B_P\right)^l\left\|\boldsymbol{X}^m\right\|_F\\&\overset{(c)}{\leq}\left(C_\sigma B_W B_P\right)^l B_X\leq\max_{1\leq l\leq L}\left(C_\sigma B_W B_P\right)^l B_X,\end{aligned}$$

where (a)–(c) results from Assumptions 5.2 and 5.3.

$$\begin{aligned}
\left\|\widetilde{\boldsymbol{H}}_{local}^{(l),m}\right\|_F &= \left\|\sigma\left(\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}\boldsymbol{W}^{(l),m}\right)\right\|_F \\
&\overset{(a)}{\leq} C_\sigma B_W \left\|\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}\right\|_F \leq C_\sigma B_W \left\|\widetilde{\boldsymbol{P}}_{local}^{(l),m}\right\| \left\|\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}\right\|_F \\
&\overset{(b)}{\leq} C_\sigma B_W B_P \left\|\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}\right\|_F \leq \left(C_\sigma B_W B_P\right)^l \left\|\boldsymbol{X}^m\right\|_F \\
&\overset{(c)}{\leq} \left(C_\sigma B_W B_P\right)^l B_X \leq \max_{1\leq l\leq L}\left(C_\sigma B_W B_P\right)^l B_X,
\end{aligned}$$

where (a)–(c) follow from Assumptions 5.2 and 5.3.

$$\begin{aligned}
\left\|\boldsymbol{H}_{full}^{(l),m}\right\|_F &= \left\|\sigma\left(\boldsymbol{P}_{full}^{(l),m}\boldsymbol{H}_{full}^{(l-1),m}\boldsymbol{W}^{(l),m}\right)\right\|_F \\
&\overset{(a)}{\leq} C_\sigma B_P B_W \left\|\boldsymbol{H}_{full}^{(l-1),m}\right\|_F \leq \left(C_\sigma B_P B_W\right)^l \left\|\boldsymbol{X}^m\right\|_F \\
&\overset{(b)}{\leq} \left(C_\sigma B_P B_W\right)^l B_X \leq \max_{1\leq l\leq L}\left(C_\sigma B_P B_W\right)^l B_X,
\end{aligned}$$

where (a) and (b) are because of Assumptions 5.2 and 5.3.

$$\begin{aligned}
\left\|\widetilde{\boldsymbol{H}}_{full}^{(l),m}\right\|_F &= \left\|\sigma\left(\left(\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m} + \widetilde{\boldsymbol{P}}_{remote}^{(l),m}\widetilde{\boldsymbol{H}}_{remote}^{(l-1),m}\right)\boldsymbol{W}^{(l),m}\right)\right\|_F \\
&\overset{(a)}{\leq} C_\sigma B_W \left\|\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m} + \widetilde{\boldsymbol{P}}_{remote}^{(l),m}\widetilde{\boldsymbol{H}}_{remote}^{(l-1),m}\right\|_F = C_\sigma B_W \left\|\widetilde{\boldsymbol{P}}_{full}^{(l),m}\widetilde{\boldsymbol{H}}_{full}^{(l-1),m}\right\|_F \\
&\overset{(b)}{\leq} C_\sigma B_W B_P \left\|\widetilde{\boldsymbol{H}}_{full}^{(l-1),m}\right\|_F \leq \left(C_\sigma B_W B_P\right)^l \left\|\boldsymbol{X}^m\right\|_F \\
&\overset{(c)}{\leq} \left(C_\sigma B_W B_P\right)^l B_X \leq \max_{1\leq l\leq L}\left(C_\sigma B_W B_P\right)^l B_X,
\end{aligned}$$

where (a)–(c) follow from Assumptions 5.2 and 5.3.

$$\begin{aligned}
\left\|\boldsymbol{D}_{local}^{(l),m}\right\|_F &= \left\|\left[\boldsymbol{P}_{local}^{(l+1),m}\right]^\top \boldsymbol{D}_{local}^{(l+1),m} \circ \nabla\sigma\left(\boldsymbol{Z}_{local}^{(l+1),m}\right)\left[\boldsymbol{W}^{(l+1),m}\right]^\top\right\|_F \\
&\overset{(a)}{\leq} B_W C_\sigma \left\|\boldsymbol{P}_{local}^{(l+1),m}\right\|_F \left\|\boldsymbol{D}_{local}^{(l+1),m}\right\|_F \\
&\overset{(b)}{\leq} B_P B_W C_\sigma \left\|\boldsymbol{D}_{local}^{(l+1),m}\right\|_F \leq \left(B_P B_W C_\sigma\right)^{L-l} \left\|\boldsymbol{D}_{local}^{(L),m}\right\|_F \\
&\overset{(c)}{\leq} \left(B_P B_W C_\sigma\right)^{L-l} C_l \leq \max_{1\leq l\leq L}\left(B_P B_W C_\sigma\right)^{L-l} C_l,
\end{aligned}$$

where (a)–(c) are because of Assumptions 5.1–5.3.

$$\begin{aligned}
\left\|\widetilde{\boldsymbol{D}}_{local}^{(l),m}\right\|_F &= \left\|\left[\widetilde{\boldsymbol{P}}_{local}^{(l+1),m}\right]^\top \widetilde{\boldsymbol{D}}_{local}^{(l+1),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{local}^{(l+1),m}\right)\left[\boldsymbol{W}^{(l+1),m}\right]^\top\right\|_F \\
&\overset{(a)}{\leq} B_W C_\sigma \left\|\widetilde{\boldsymbol{P}}_{local}^{(l+1),m}\right\|_F \left\|\widetilde{\boldsymbol{D}}_{local}^{(l+1),m}\right\|_F \\
&\overset{(b)}{\leq} B_P B_W C_\sigma \left\|\widetilde{\boldsymbol{D}}_{local}^{(l+1),m}\right\|_F \leq \left(B_P B_W C_\sigma\right)^{L-l} \left\|\widetilde{\boldsymbol{D}}_{local}^{(L),m}\right\|_F \\
&\overset{(c)}{\leq} \left(B_P B_W C_\sigma\right)^{L-l} C_l \leq \max_{1\leq l\leq L}\left(B_P B_W C_\sigma\right)^{L-l} C_l,
\end{aligned}$$

where (a)–(c) follow from Assumptions 5.1–5.3.

$$\begin{aligned}\left\|\boldsymbol{D}_{full}^{(l),m}\right\|_F &= \left\|\left[\boldsymbol{P}_{full}^{(l+1),m}\right]^\top \boldsymbol{D}_{full}^{(l+1),m} \circ \nabla\sigma\left(\boldsymbol{Z}_{full}^{(l+1),m}\right)\left[\boldsymbol{W}^{(l+1),m}\right]^\top\right\|_F \\ &\overset{(a)}{\leq} B_P B_W C_\sigma \left\|\boldsymbol{D}_{full}^{(l+1),m}\right\|_F \leq \left(B_P B_W C_\sigma\right)^{L-l}\left\|\boldsymbol{D}_{full}^{(L),m}\right\|_F \\ &\overset{(b)}{\leq} \left(B_P B_W C_\sigma\right)^{L-l} C_l \leq \max_{1\leq l\leq L}\left(B_P B_W C_\sigma\right)^{L-l} C_l,\end{aligned}$$

where (a) and (b) use Assumptions 5.1–5.3.

$$\begin{aligned}\left\|\widetilde{\boldsymbol{D}}_{full}^{(l),m}\right\|_F &= \left\|\left[\widetilde{\boldsymbol{P}}_{local}^{(l+1),m} + \widetilde{\boldsymbol{P}}_{remote}^{(l+1),m}\right]^\top \widetilde{\boldsymbol{D}}_{full}^{(l+1),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l+1),m}\right)\left[\boldsymbol{W}^{(l+1),m}\right]^\top\right\|_F \\ &= \left\|\left[\widetilde{\boldsymbol{P}}_{full}^{(l+1),m}\right]^\top \widetilde{\boldsymbol{D}}_{full}^{(l+1),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l+1),m}\right)\left[\boldsymbol{W}^{(l+1),m}\right]^\top\right\|_F \\ &\overset{(a)}{\leq} B_P B_W C_\sigma \left\|\widetilde{\boldsymbol{D}}_{full}^{(l+1),m}\right\|_F \leq \left(B_P B_W C_\sigma\right)^{L-l}\left\|\widetilde{\boldsymbol{D}}_{full}^{(L),m}\right\|_F \\ &\overset{(b)}{\leq} \left(B_P B_W C_\sigma\right)^{L-l} C_l \leq \max_{1\leq l\leq L}\left(B_P B_W C_\sigma\right)^{L-l} C_l,\end{aligned}$$

where (a) and (b) utilize Assumptions 5.1–5.3.

□

D.4 Errors Induced by Sampling

Lemma D.4. *Under Assumptions 5.1–5.3, and for any $l \in [L]$, the errors caused by sampling are bounded, i.e.,*

$$\begin{aligned}&\left\|\widetilde{\boldsymbol{H}}_{local}^{(l),m} - \boldsymbol{H}_{local}^{(l),m}\right\|_F \leq B_{\Delta H}^l, && \left\|\widetilde{\boldsymbol{H}}_{full}^{(l),m} - \boldsymbol{H}_{full}^{(l),m}\right\|_F \leq B_{\Delta H}^f, \\ &\left\|\widetilde{\boldsymbol{D}}_{local}^{(l),m} - \boldsymbol{D}_{local}^{(l),m}\right\|_F \leq B_{\Delta D}^l, && \left\|\widetilde{\boldsymbol{D}}_{full}^{(l),m} - \boldsymbol{D}_{full}^{(l),m}\right\|_F \leq B_{\Delta D}^f,\end{aligned}$$

where

$$\begin{aligned}B_{\Delta H}^l &= \max_{1\leq l\leq L}\left(\left(C_\sigma B_W B_H^l B_{\Delta P}^l\right)^l + \left(C_\sigma B_W B_P\right)^l B_{\Delta X}^l\right), \\ B_{\Delta H}^f &= \max_{1\leq l\leq L}\left(\left(C_\sigma B_W B_H^f B_{\Delta P}^f\right)^l + \left(C_\sigma B_W B_P\right)^l B_{\Delta X}^f\right), \\ B_{\Delta D}^l &= \max_{1\leq l\leq L}\Bigg(\left(B_W B_D^l C_\sigma B_{\Delta P}^l + B_W^2 B_P B_D^l L_\sigma B_H^l B_{\Delta P}^l + B_W^2 B_P^2 B_D^l L_\sigma B_{\Delta H}^l\right)^{L-l} \\ &\qquad + \left(B_W B_P C_\sigma\right)^{L-l} L_l B_{\Delta H}^l\Bigg), \\ B_{\Delta D}^f &= \max_{1\leq l\leq L}\Bigg(\left(B_W B_D^f C_\sigma B_{\Delta P}^f + B_W^2 B_P B_D^f L_\sigma B_H^f B_{\Delta P}^f + B_W^2 B_P^2 B_D^f L_\sigma B_{\Delta H}^f\right)^{L-l} \\ &\qquad + \left(B_W B_P C_\sigma\right)^{L-l} L_l B_{\Delta H}^f\Bigg).\end{aligned}$$

Proof.

$$\begin{aligned}&\left\|\widetilde{\boldsymbol{H}}_{local}^{(l),m} - \boldsymbol{H}_{local}^{(l),m}\right\|_F \\ &= \left\|\sigma\left(\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}\boldsymbol{W}^{(l),m}\right) - \sigma\left(\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}\right)\boldsymbol{W}^{(l),m}\right\|_F\end{aligned}$$

$$
\begin{aligned}
&\overset{(a)}{\leq} C_\sigma B_W \left\| \widetilde{\boldsymbol{P}}_{local}^{(l),m} \widetilde{\boldsymbol{H}}_{local}^{(l-1),m} - \boldsymbol{P}_{local}^{(l),m} \boldsymbol{H}_{local}^{(l-1),m} \right\|_F \\
&\leq C_\sigma B_W \left\| \widetilde{\boldsymbol{P}}_{local}^{(l),m} \widetilde{\boldsymbol{H}}_{local}^{(l-1),m} - \boldsymbol{P}_{local}^{(l),m} \widetilde{\boldsymbol{H}}_{local}^{(l-1),m} \right\|_F + C_\sigma B_W \left\| \boldsymbol{P}_{local}^{(l),m} \widetilde{\boldsymbol{H}}_{local}^{(l-1),m} - \boldsymbol{P}_{local}^{(l),m} \boldsymbol{H}_{local}^{(l-1),m} \right\|_F \\
&\overset{(b)}{\leq} C_\sigma B_W B_H^l \left\| \widetilde{\boldsymbol{P}}_{local}^{(l),m} - \boldsymbol{P}_{local}^{(l),m} \right\|_F + C_\sigma B_W B_P \left\| \widetilde{\boldsymbol{H}}_{local}^{(l-1),m} - \boldsymbol{H}_{local}^{(l-1),m} \right\|_F \\
&\overset{(c)}{\leq} C_\sigma B_W B_H^l B_{\Delta P}^l + C_\sigma B_W B_P \left\| \widetilde{\boldsymbol{H}}_{local}^{(l-1),m} - \boldsymbol{H}_{local}^{(l-1),m} \right\|_F \\
&\leq \left(C_\sigma B_W B_H^l B_{\Delta P}^l \right)^l + \left(C_\sigma B_W B_P \right)^l \left\| \widetilde{\boldsymbol{X}}_{local}^m - \boldsymbol{X}_{local}^m \right\|_F \\
&\overset{(d)}{\leq} \left(C_\sigma B_W B_H^l B_{\Delta P}^l \right)^l + \left(C_\sigma B_W B_P \right)^l B_{\Delta X}^l \\
&\leq \max_{1 \leq l \leq L} \left(\left(C_\sigma B_W B_H^l B_{\Delta P}^l \right)^l + \left(C_\sigma B_W B_P \right)^l B_{\Delta X}^l \right),
\end{aligned} \tag{12}
$$

where (a) uses Assumptions 5.2 and 5.3, (b) is because of Assumption 5.3 and Lemma D.3, and (c) and (d) follow from Proposition D.1.

$$
\begin{aligned}
&\left\| \widetilde{\boldsymbol{H}}_{full}^{(l),m} - \boldsymbol{H}_{full}^{(l),m} \right\|_F \\
&= \left\| \sigma \left(\left(\widetilde{\boldsymbol{P}}_{local}^{(l),m} \widetilde{\boldsymbol{H}}_{local}^{(l-1),m} + \widetilde{\boldsymbol{P}}_{remote}^{(l),m} \widetilde{\boldsymbol{H}}_{remote}^{(l-1),m} \right) \boldsymbol{W}^{(l),m} \right) - \sigma \left(\boldsymbol{P}_{full}^{(l),m} \boldsymbol{H}_{full}^{(l-1),m} \right) \boldsymbol{W}^{(l),m} \right\|_F \\
&\overset{(a)}{\leq} C_\sigma B_W \left\| \widetilde{\boldsymbol{P}}_{full}^{(l),m} \widetilde{\boldsymbol{H}}_{full}^{(l-1),m} - \boldsymbol{P}_{full}^{(l),m} \boldsymbol{H}_{full}^{(l-1),m} \right\|_F \\
&\leq C_\sigma B_W \left\| \widetilde{\boldsymbol{P}}_{full}^{(l),m} \widetilde{\boldsymbol{H}}_{full}^{(l-1),m} - \boldsymbol{P}_{full}^{(l),m} \widetilde{\boldsymbol{H}}_{full}^{(l-1),m} \right\|_F + C_\sigma B_W \left\| \boldsymbol{P}_{full}^{(l),m} \widetilde{\boldsymbol{H}}_{full}^{(l-1),m} - \boldsymbol{P}_{full}^{(l),m} \boldsymbol{H}_{full}^{(l-1),m} \right\|_F \\
&\overset{(b)}{\leq} C_\sigma B_W B_H^f \left\| \widetilde{\boldsymbol{P}}_{full}^{(l),m} - \boldsymbol{P}_{full}^{(l),m} \right\|_F + C_\sigma B_W B_P \left\| \widetilde{\boldsymbol{H}}_{full}^{(l-1),m} - \boldsymbol{H}_{full}^{(l-1),m} \right\|_F \\
&\overset{(c)}{\leq} C_\sigma B_W B_H^f B_{\Delta P}^f + C_\sigma B_W B_P \left\| \widetilde{\boldsymbol{H}}_{full}^{(l-1),m} - \boldsymbol{H}_{full}^{(l-1),m} \right\|_F \\
&\leq \left(C_\sigma B_W B_H^f B_{\Delta P}^f \right)^l + \left(C_\sigma B_W B_P \right)^l \left\| \widetilde{\boldsymbol{X}}_{full}^m - \boldsymbol{X}_{full}^m \right\|_F \\
&\overset{(d)}{\leq} \left(C_\sigma B_W B_H^f B_{\Delta P}^f \right)^l + \left(C_\sigma B_W B_P \right)^l B_{\Delta X}^f \\
&\leq \max_{1 \leq l \leq L} \left(\left(C_\sigma B_W B_H^f B_{\Delta P}^f \right)^l + \left(C_\sigma B_W B_P \right)^l B_{\Delta X}^f \right),
\end{aligned} \tag{13}
$$

where (a) follows from Assumptions 5.2 and 5.3, (b) is due to Assumption 5.3 and Lemma D.3, and (c) and (d) are because of Proposition D.1.

$$
\begin{aligned}
&\left\| \widetilde{\boldsymbol{D}}_{local}^{(l),m} - \boldsymbol{D}_{local}^{(l),m} \right\|_F \\
&= \Bigg\| \left[\widetilde{\boldsymbol{P}}_{local}^{(l+1),m} \right]^\top \widetilde{\boldsymbol{D}}_{local}^{(l+1),m} \circ \nabla \sigma \left(\widetilde{\boldsymbol{Z}}_{local}^{(l+1),m} \right) \left[\boldsymbol{W}^{(l+1),m} \right]^\top \\
&\qquad\qquad - \left[\boldsymbol{P}_{local}^{(l+1),m} \right]^\top \boldsymbol{D}_{local}^{(l+1),m} \circ \nabla \sigma \left(\boldsymbol{Z}_{local}^{(l+1),m} \right) \left[\boldsymbol{W}^{(l+1),m} \right]^\top \Bigg\|_F \\
&\overset{(a)}{\leq} B_W \left\| \left[\widetilde{\boldsymbol{P}}_{local}^{(l+1),m} \right]^\top \widetilde{\boldsymbol{D}}_{local}^{(l+1),m} \circ \nabla \sigma \left(\widetilde{\boldsymbol{Z}}_{local}^{(l+1),m} \right) - \left[\boldsymbol{P}_{local}^{(l+1),m} \right]^\top \boldsymbol{D}_{local}^{(l+1),m} \circ \nabla \sigma \left(\boldsymbol{Z}_{local}^{(l+1),m} \right) \right\|_F \\
&\leq B_W \left\| \left[\widetilde{\boldsymbol{P}}_{local}^{(l+1),m} \right]^\top \widetilde{\boldsymbol{D}}_{local}^{(l+1),m} \circ \nabla \sigma \left(\widetilde{\boldsymbol{Z}}_{local}^{(l+1),m} \right) - \left[\boldsymbol{P}_{local}^{(l+1),m} \right]^\top \widetilde{\boldsymbol{D}}_{local}^{(l+1),m} \circ \nabla \sigma \left(\widetilde{\boldsymbol{Z}}_{local}^{(l+1),m} \right) \right\|_F \\
&+ B_W \left\| \left[\boldsymbol{P}_{local}^{(l+1),m} \right]^\top \widetilde{\boldsymbol{D}}_{local}^{(l+1),m} \circ \nabla \sigma \left(\widetilde{\boldsymbol{Z}}_{local}^{(l+1),m} \right) - \left[\boldsymbol{P}_{local}^{(l+1),m} \right]^\top \boldsymbol{D}_{local}^{(l+1),m} \circ \nabla \sigma \left(\widetilde{\boldsymbol{Z}}_{local}^{(l+1),m} \right) \right\|_F
\end{aligned}
$$

$$
\begin{aligned}
&+ B_W \left\| \left[\boldsymbol{P}_{local}^{(l+1),m}\right]^\top \boldsymbol{D}_{local}^{(l+1),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{local}^{(l+1),m}\right) - \left[\boldsymbol{P}_{local}^{(l+1),m}\right]^\top \boldsymbol{D}_{local}^{(l+1),m} \circ \nabla\sigma\left(\boldsymbol{Z}_{local}^{(l+1),m}\right) \right\|_F \\
&\overset{(b)}{\leq} B_W B_D^l C_\sigma \left\| \widetilde{\boldsymbol{P}}_{local}^{(l+1),m} - \boldsymbol{P}_{local}^{(l+1),m} \right\|_F + B_W B_P C_\sigma \left\| \widetilde{\boldsymbol{D}}_{local}^{(l+1),m} - \boldsymbol{D}_{local}^{(l+1),m} \right\|_F \\
&+ B_W B_P B_D^l \left\| \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{local}^{(l+1),m}\right) - \nabla\sigma\left(\boldsymbol{Z}_{local}^{(l+1),m}\right) \right\|_F \\
&\overset{(c)}{\leq} B_W B_D^l C_\sigma \left\| \widetilde{\boldsymbol{P}}_{local}^{(l+1),m} - \boldsymbol{P}_{local}^{(l+1),m} \right\|_F + B_W B_P C_\sigma \left\| \widetilde{\boldsymbol{D}}_{local}^{(l+1),m} - \boldsymbol{D}_{local}^{(l+1),m} \right\|_F \\
&+ B_W^2 B_P B_D^l L_\sigma \left\| \widetilde{\boldsymbol{P}}_{local}^{(l),m} \widetilde{\boldsymbol{H}}_{local}^{(l-1),m} - \boldsymbol{P}_{local}^{(l),m} \boldsymbol{H}_{local}^{(l-1),m} \right\|_F \\
&\leq B_W B_D^l C_\sigma \left\| \widetilde{\boldsymbol{P}}_{local}^{(l+1),m} - \boldsymbol{P}_{local}^{(l+1),m} \right\|_F + B_W B_P C_\sigma \left\| \widetilde{\boldsymbol{D}}_{local}^{(l+1),m} - \boldsymbol{D}_{local}^{(l+1),m} \right\|_F \\
&+ B_W^2 B_P B_D^l L_\sigma \left\| \widetilde{\boldsymbol{P}}_{local}^{(l),m} \widetilde{\boldsymbol{H}}_{local}^{(l-1),m} - \boldsymbol{P}_{local}^{(l),m} \widetilde{\boldsymbol{H}}_{local}^{(l-1),m} \right\|_F \\
&+ B_W^2 B_P B_D^l L_\sigma \left\| \boldsymbol{P}_{local}^{(l),m} \widetilde{\boldsymbol{H}}_{local}^{(l-1),m} - \boldsymbol{P}_{local}^{(l),m} \boldsymbol{H}_{local}^{(l-1),m} \right\|_F \\
&\overset{(d)}{\leq} B_W B_D^l C_\sigma \left\| \widetilde{\boldsymbol{P}}_{local}^{(l+1),m} - \boldsymbol{P}_{local}^{(l+1),m} \right\|_F + B_W B_P C_\sigma \left\| \widetilde{\boldsymbol{D}}_{local}^{(l+1),m} - \boldsymbol{D}_{local}^{(l+1),m} \right\|_F \\
&+ B_W^2 B_P B_D^l L_\sigma B_H^l \left\| \widetilde{\boldsymbol{P}}_{local}^{(l),m} - \boldsymbol{P}_{local}^{(l),m} \right\|_F + B_W^2 B_P^2 B_D^l L_\sigma \left\| \widetilde{\boldsymbol{H}}_{local}^{(l-1),m} - \boldsymbol{H}_{local}^{(l-1),m} \right\|_F \\
&\overset{(e)}{\leq} B_W B_D^l C_\sigma B_{\Delta P}^l + B_W^2 B_P B_D^l L_\sigma B_H^l B_{\Delta P}^l + B_W^2 B_P^2 B_D^l L_\sigma B_{\Delta H}^l \\
&+ B_W B_P C_\sigma \left\| \widetilde{\boldsymbol{D}}_{local}^{(l+1),m} - \boldsymbol{D}_{local}^{(l+1),m} \right\|_F \\
&\leq \left(B_W B_D^l C_\sigma B_{\Delta P}^l + B_W^2 B_P B_D^l L_\sigma B_H^l B_{\Delta P}^l + B_W^2 B_P^2 B_D^l L_\sigma B_{\Delta H}^l\right)^{L-l} \\
&+ \left(B_W B_P C_\sigma\right)^{L-l} \left\| \widetilde{\boldsymbol{D}}_{local}^{(L),m} - \boldsymbol{D}_{local}^{(L),m} \right\|_F \\
&\overset{(f)}{\leq} \left(B_W B_D^l C_\sigma B_{\Delta P}^l + B_W^2 B_P B_D^l L_\sigma B_H^l B_{\Delta P}^l + B_W^2 B_P^2 B_D^l L_\sigma B_{\Delta H}^l\right)^{L-l} \\
&+ \left(B_W B_P C_\sigma\right)^{L-l} L_l \left\| \widetilde{\boldsymbol{H}}_{local}^{(L),m} - \boldsymbol{H}_{local}^{(L),m} \right\|_F \\
&\overset{(g)}{\leq} \left(B_W B_D^l C_\sigma B_{\Delta P}^l + B_W^2 B_P B_D^l L_\sigma B_H^l B_{\Delta P}^l + B_W^2 B_P^2 B_D^l L_\sigma B_{\Delta H}^l\right)^{L-l} \\
&+ \left(B_W B_P C_\sigma\right)^{L-l} L_l B_{\Delta H}^l \\
&\leq \max_{1\leq l\leq L} \Big(\left(B_W B_D^l C_\sigma B_{\Delta P}^l + B_W^2 B_P B_D^l L_\sigma B_H^l B_{\Delta P}^l + B_W^2 B_P^2 B_D^l L_\sigma B_{\Delta H}^l\right)^{L-l} \\
&+ \left(B_W B_P C_\sigma\right)^{L-l} L_l B_{\Delta H}^l \Big),
\end{aligned}
$$

where (a) uses Assumption 5.3, (b) is because of Assumptions 5.2 and 5.3 and Lemma D.3, (c) follows from Assumptions 5.2 and 5.3, (d) utilizes Assumption 5.3 and Lemma D.3, (e) results from Eq. (12) and Proposition D.1, (f) is because of Assumption 5.1, and (g) is due to Eq. (12).

$$
\begin{aligned}
&\left\| \widetilde{\boldsymbol{D}}_{full}^{(l),m} - \boldsymbol{D}_{full}^{(l),m} \right\|_F \\
&= \Bigg\| \left[\widetilde{\boldsymbol{P}}_{local}^{(l+1),m} + \widetilde{\boldsymbol{P}}_{remote}^{(l+1),m}\right]^\top \widetilde{\boldsymbol{D}}_{full}^{(l+1),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l+1),m}\right) \left[\boldsymbol{W}^{(l+1),m}\right]^\top \\
&- \left[\boldsymbol{P}_{full}^{(l+1),m}\right]^\top \boldsymbol{D}_{full}^{(l+1),m} \circ \nabla\sigma\left(\boldsymbol{Z}_{full}^{(l+1),m}\right) \left[\boldsymbol{W}^{(l+1),m}\right]^\top \Bigg\|_F \\
&\overset{(a)}{\leq} B_W \left\| \left[\widetilde{\boldsymbol{P}}_{full}^{(l+1),m}\right]^\top \widetilde{\boldsymbol{D}}_{full}^{(l+1),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l+1),m}\right) - \left[\boldsymbol{P}_{full}^{(l+1),m}\right]^\top \boldsymbol{D}_{full}^{(l+1),m} \circ \nabla\sigma\left(\boldsymbol{Z}_{full}^{(l+1),m}\right) \right\|_F \\
&\leq B_W \left\| \left[\widetilde{\boldsymbol{P}}_{full}^{(l+1),m}\right]^\top \widetilde{\boldsymbol{D}}_{full}^{(l+1),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l+1),m}\right) - \left[\boldsymbol{P}_{full}^{(l+1),m}\right]^\top \widetilde{\boldsymbol{D}}_{full}^{(l+1),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l+1),m}\right) \right\|_F
\end{aligned}
$$

$$
\begin{aligned}
&+ B_W \left\| \left[\boldsymbol{P}_{full}^{(l+1),m}\right]^\top \widetilde{\boldsymbol{D}}_{full}^{(l+1),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l+1),m}\right) - \left[\boldsymbol{P}_{full}^{(l+1),m}\right]^\top \boldsymbol{D}_{full}^{(l+1),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l+1),m}\right) \right\|_F \\
&+ B_W \left\| \left[\boldsymbol{P}_{full}^{(l+1),m}\right]^\top \boldsymbol{D}_{full}^{(l+1),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l+1),m}\right) - \left[\boldsymbol{P}_{full}^{(l+1),m}\right]^\top \boldsymbol{D}_{full}^{(l+1),m} \circ \nabla\sigma\left(\boldsymbol{Z}_{full}^{(l+1),m}\right) \right\|_F \\
&\overset{(b)}{\leq} B_W B_D^f C_\sigma \left\| \widetilde{\boldsymbol{P}}_{full}^{(l+1),m} - \boldsymbol{P}_{full}^{(l+1),m} \right\|_F + B_W B_P C_\sigma \left\| \widetilde{\boldsymbol{D}}_{full}^{(l+1),m} - \boldsymbol{D}_{full}^{(l+1),m} \right\|_F \\
&+ B_W B_P B_D^f \left\| \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l+1),m}\right) - \nabla\sigma\left(\boldsymbol{Z}_{full}^{(l+1),m}\right) \right\|_F \\
&\overset{(c)}{\leq} B_W B_D^f C_\sigma \left\| \widetilde{\boldsymbol{P}}_{full}^{(l+1),m} - \boldsymbol{P}_{full}^{(l+1),m} \right\|_F + B_W B_P C_\sigma \left\| \widetilde{\boldsymbol{D}}_{full}^{(l+1),m} - \boldsymbol{D}_{full}^{(l+1),m} \right\|_F \\
&+ B_W^2 B_P B_D^f L_\sigma \left\| \widetilde{\boldsymbol{P}}_{full}^{(l+1),m} \widetilde{\boldsymbol{H}}_{full}^{(l),m} - \boldsymbol{P}_{full}^{(l+1),m} \boldsymbol{H}_{full}^{(l),m} \right\|_F \\
&\leq B_W B_D^f C_\sigma \left\| \widetilde{\boldsymbol{P}}_{full}^{(l+1),m} - \boldsymbol{P}_{full}^{(l+1),m} \right\|_F + B_W B_P C_\sigma \left\| \widetilde{\boldsymbol{D}}_{full}^{(l+1),m} - \boldsymbol{D}_{full}^{(l+1),m} \right\|_F \\
&+ B_W^2 B_P B_D^f L_\sigma \left\| \widetilde{\boldsymbol{P}}_{full}^{(l+1),m} \widetilde{\boldsymbol{H}}_{full}^{(l),m} - \boldsymbol{P}_{full}^{(l+1),m} \widetilde{\boldsymbol{H}}_{full}^{(l),m} \right\|_F \\
&+ B_W^2 B_P B_D^f L_\sigma \left\| \boldsymbol{P}_{full}^{(l+1),m} \widetilde{\boldsymbol{H}}_{full}^{(l),m} - \boldsymbol{P}_{full}^{(l+1),m} \boldsymbol{H}_{full}^{(l),m} \right\|_F \\
&\overset{(d)}{\leq} B_W B_D^f C_\sigma \left\| \widetilde{\boldsymbol{P}}_{full}^{(l+1),m} - \boldsymbol{P}_{full}^{(l+1),m} \right\|_F + B_W B_P C_\sigma \left\| \widetilde{\boldsymbol{D}}_{full}^{(l+1),m} - \boldsymbol{D}_{full}^{(l+1),m} \right\|_F \\
&+ B_W^2 B_P B_D^f L_\sigma B_H^f \left\| \widetilde{\boldsymbol{P}}_{full}^{(l+1),m} - \boldsymbol{P}_{full}^{(l+1),m} \right\|_F + B_W^2 B_P^2 B_D^f L_\sigma \left\| \widetilde{\boldsymbol{H}}_{full}^{(l),m} - \boldsymbol{H}_{full}^{(l),m} \right\|_F \\
&\overset{(e)}{\leq} B_W B_D^f C_\sigma B_{\Delta P}^f + B_W^2 B_P B_D^f L_\sigma B_H^f B_{\Delta P}^f + B_W^2 B_P^2 B_D^f L_\sigma B_{\Delta H}^f \\
&+ B_W B_P C_\sigma \left\| \widetilde{\boldsymbol{D}}_{full}^{(l+1),m} - \boldsymbol{D}_{full}^{(l+1),m} \right\|_F \\
&\leq \left(B_W B_D^f C_\sigma B_{\Delta P}^f + B_W^2 B_P B_D^f L_\sigma B_H^f B_{\Delta P}^f + B_W^2 B_P^2 B_D^f L_\sigma B_{\Delta H}^f \right)^{L-l} \\
&+ \left(B_W B_P C_\sigma\right)^{L-l} \left\| \widetilde{\boldsymbol{D}}_{full}^{(L),m} - \boldsymbol{D}_{full}^{(L),m} \right\|_F \\
&\overset{(f)}{\leq} \left(B_W B_D^f C_\sigma B_{\Delta P}^f + B_W^2 B_P B_D^f L_\sigma B_H^f B_{\Delta P}^f + B_W^2 B_P^2 B_D^f L_\sigma B_{\Delta H}^f \right)^{L-l} \\
&+ \left(B_W B_P C_\sigma\right)^{L-l} L_l \left\| \widetilde{\boldsymbol{H}}_{full}^{(L),m} - \boldsymbol{H}_{full}^{(L),m} \right\|_F \\
&\overset{(g)}{\leq} \left(B_W B_D^f C_\sigma B_{\Delta P}^f + B_W^2 B_P B_D^f L_\sigma B_H^f B_{\Delta P}^f + B_W^2 B_P^2 B_D^f L_\sigma B_{\Delta H}^f \right)^{L-l} \\
&+ \left(B_W B_P C_\sigma\right)^{L-l} L_l B_{\Delta H}^f \\
&\leq \max_{1\leq l\leq L} \Bigg(\left(B_W B_D^f C_\sigma B_{\Delta P}^f + B_W^2 B_P B_D^f L_\sigma B_H^f B_{\Delta P}^f + B_W^2 B_P^2 B_D^f L_\sigma B_{\Delta H}^f \right)^{L-l} \\
&+ \left(B_W B_P C_\sigma\right)^{L-l} L_l B_{\Delta H}^f \Bigg),
\end{aligned}
$$

where (a) is because of Assumption 5.3, (b) results from Assumptions 5.2 and 5.3 and Lemma D.3, (c) uses Assumptions 5.2 and 5.3, (d) is due to Assumption 5.3 and Lemma D.3, (e) follows from Eq. (13) and Proposition D.1, (f) utilizes Assumption 5.1, and (g) is because of Eq. (13).

□

D.5 Errors Induced by Missing Cross-Client Neighbors

Lemma D.5. *Under Assumptions 5.1–5.3, and for any $l \in [L]$, the errors caused by the information loss of the cross-client neighbors are bounded, i.e.,*

$$
\left\| \boldsymbol{H}_{local}^{(l),m} - \boldsymbol{H}_{full}^{(l),m} \right\|_F \leq B_{\Delta H}^r, \qquad \left\| \boldsymbol{D}_{local}^{(l),m} - \boldsymbol{D}_{full}^{(l),m} \right\|_F \leq B_{\Delta D}^r,
$$

where

$$B^r_{\Delta H} = \max_{1\le l\le L}\left(\left(C_\sigma B_W B_P\right)^l B^r_X + \left(C_\sigma B_W B^f_H B_P\right)^l\right),$$

$$B^r_{\Delta D} = \max_{1\le l\le L}\Bigg(\left(B_W B^l_D C_\sigma B_P + B^2_W B^2_P B^f_D L_\sigma B^l_H + B^2_W B^2_P B^f_D L_\sigma B^r_{\Delta H}\right)^{L-l} + \left(B_W B_P C\sigma\right)^{L-l} L_l B^r_{\Delta H}\Bigg).$$

Proof.

$$\begin{aligned}
\left\|\boldsymbol{H}^{(l),m}_{local} - \boldsymbol{H}^{(l),m}_{full}\right\|_F &= \left\|\sigma\left(\boldsymbol{P}^{(l),m}_{local}\boldsymbol{H}^{(l-1),m}_{local}\right)\boldsymbol{W}^{(l),m} - \sigma\left(\boldsymbol{P}^{(l),m}_{full}\boldsymbol{H}^{(l-1),m}_{full}\right)\boldsymbol{W}^{(l),m}\right\|_F \\
&\overset{(a)}{\le} C_\sigma B_W\left\|\boldsymbol{P}^{(l),m}_{local}\boldsymbol{H}^{(l-1),m}_{local} - \boldsymbol{P}^{(l),m}_{full}\boldsymbol{H}^{(l-1),m}_{full}\right\|_F \\
&\le C_\sigma B_W\left\|\boldsymbol{P}^{(l),m}_{local}\boldsymbol{H}^{(l-1),m}_{local} - \boldsymbol{P}^{(l),m}_{local}\boldsymbol{H}^{(l-1),m}_{full}\right\|_F \\
&+ C_\sigma B_W\left\|\boldsymbol{P}^{(l),m}_{local}\boldsymbol{H}^{(l-1),m}_{full} - \boldsymbol{P}^{(l),m}_{full}\boldsymbol{H}^{(l-1),m}_{full}\right\|_F \\
&\overset{(b)}{\le} C_\sigma B_W B_P\left\|\boldsymbol{H}^{(l-1),m}_{local} - \boldsymbol{H}^{(l-1),m}_{full}\right\|_F + C_\sigma B_W B^f_H\left\|\boldsymbol{P}^{(l),m}_{local} - \boldsymbol{P}^{(l),m}_{full}\right\|_F \\
&\le C_\sigma B_W B_P\left\|\boldsymbol{H}^{(l-1),m}_{local} - \boldsymbol{H}^{(l-1),m}_{full}\right\|_F + C_\sigma B_W B^f_H\left\|\boldsymbol{P}^{(l),m}_{remote}\right\|_F \\
&\overset{(c)}{\le} C_\sigma B_W B_P\left\|\boldsymbol{H}^{(l-1),m}_{local} - \boldsymbol{H}^{(l-1),m}_{full}\right\|_F + C_\sigma B_W B^f_H B_P \\
&\le \left(C_\sigma B_W B_P\right)^l\left\|\boldsymbol{X}^m_{local} - \boldsymbol{X}^m_{full}\right\|_F + \left(C_\sigma B_W B^f_H B_P\right)^l \\
&\overset{(d)}{\le} \left(C_\sigma B_W B_P\right)^l B^r_X + \left(C_\sigma B_W B^f_H B_P\right)^l \\
&\le \max_{1\le l\le L}\left(\left(C_\sigma B_W B_P\right)^l B^r_X + \left(C_\sigma B_W B^f_H B_P\right)^l\right),
\end{aligned} \tag{14}$$

where (a) uses Assumptions 5.2 and 5.3, (b) is because of Assumption 5.3 and Lemma D.3, (c) follows from Assumption 5.3, and (d) is due to Proposition D.1.

$$\begin{aligned}
&\left\|\boldsymbol{D}^{(l),m}_{local} - \boldsymbol{D}^{(l),m}_{full}\right\|_F \\
&= \Bigg\|\left[\boldsymbol{P}^{(l+1),m}_{local}\right]^\top \boldsymbol{D}^{(l+1),m}_{local}\circ\nabla\sigma\left(\boldsymbol{Z}^{(l+1),m}_{local}\right)\left[\boldsymbol{W}^{(l+1),m}\right]^\top \\
&- \left[\boldsymbol{P}^{(l+1),m}_{full}\right]^\top \boldsymbol{D}^{(l+1),m}_{full}\circ\nabla\sigma\left(\boldsymbol{Z}^{(l+1),m}_{full}\right)\left[\boldsymbol{W}^{(l+1),m}\right]^\top\Bigg\|_F \\
&\overset{(a)}{\le} B_W\left\|\left[\boldsymbol{P}^{(l+1),m}_{local}\right]^\top \boldsymbol{D}^{(l+1),m}_{local}\circ\nabla\sigma\left(\boldsymbol{Z}^{(l+1),m}_{local}\right) - \left[\boldsymbol{P}^{(l+1),m}_{full}\right]^\top \boldsymbol{D}^{(l+1),m}_{full}\circ\nabla\sigma\left(\boldsymbol{Z}^{(l+1),m}_{full}\right)\right\|_F \\
&\le B_W\left\|\left[\boldsymbol{P}^{(l+1),m}_{local}\right]^\top \boldsymbol{D}^{(l+1),m}_{local}\circ\nabla\sigma\left(\boldsymbol{Z}^{(l+1),m}_{local}\right) - \left[\boldsymbol{P}^{(l+1),m}_{full}\right]^\top \boldsymbol{D}^{(l+1),m}_{local}\circ\nabla\sigma\left(\boldsymbol{Z}^{(l+1),m}_{local}\right)\right\|_F \\
&+ B_W\left\|\left[\boldsymbol{P}^{(l+1),m}_{full}\right]^\top \boldsymbol{D}^{(l+1),m}_{local}\circ\nabla\sigma\left(\boldsymbol{Z}^{(l+1),m}_{local}\right) - \left[\boldsymbol{P}^{(l+1),m}_{full}\right]^\top \boldsymbol{D}^{(l+1),m}_{full}\circ\nabla\sigma\left(\boldsymbol{Z}^{(l+1),m}_{local}\right)\right\|_F \\
&+ B_W\left\|\left[\boldsymbol{P}^{(l+1),m}_{full}\right]^\top \boldsymbol{D}^{(l+1),m}_{full}\circ\nabla\sigma\left(\boldsymbol{Z}^{(l+1),m}_{local}\right) - \left[\boldsymbol{P}^{(l+1),m}_{full}\right]^\top \boldsymbol{D}^{(l+1),m}_{full}\circ\nabla\sigma\left(\boldsymbol{Z}^{(l+1),m}_{full}\right)\right\|_F \\
&\overset{(b)}{\le} B_W B^l_D C_\sigma\left\|\boldsymbol{P}^{(l+1),m}_{local} - \boldsymbol{P}^{(l+1),m}_{full}\right\|_F + B_W B_P C\sigma\left\|\boldsymbol{D}^{(l+1),m}_{local} - \boldsymbol{D}^{(l+1),m}_{full}\right\|_F \\
&+ B_W B_P B^f_D\left\|\nabla\sigma\left(\boldsymbol{Z}^{(l+1),m}_{local}\right) - \nabla\sigma\left(\boldsymbol{Z}^{(l+1),m}_{full}\right)\right\|_F
\end{aligned}$$

$$
\begin{aligned}
&\overset{(c)}{\leq} B_W B_D^l C_\sigma \left\| \boldsymbol{P}_{local}^{(l+1),m} - \boldsymbol{P}_{full}^{(l+1),m} \right\|_F + B_W B_P C\sigma \left\| \boldsymbol{D}_{local}^{(l+1),m} - \boldsymbol{D}_{full}^{(l+1),m} \right\|_F \\
&+ B_W^2 B_P B_D^f L_\sigma \left\| \boldsymbol{P}_{local}^{(l+1),m} \boldsymbol{H}_{local}^{(l),m} - \boldsymbol{P}_{full}^{(l+1),m} \boldsymbol{H}_{full}^{(l),m} \right\|_F \\
&\leq B_W B_D^l C_\sigma \left\| \boldsymbol{P}_{local}^{(l+1),m} - \boldsymbol{P}_{full}^{(l+1),m} \right\|_F + B_W B_P C\sigma \left\| \boldsymbol{D}_{local}^{(l+1),m} - \boldsymbol{D}_{full}^{(l+1),m} \right\|_F \\
&+ B_W^2 B_P B_D^f L_\sigma \left\| \boldsymbol{P}_{local}^{(l+1),m} \boldsymbol{H}_{local}^{(l),m} - \boldsymbol{P}_{full}^{(l+1),m} \boldsymbol{H}_{local}^{(l),m} \right\|_F \\
&+ B_W^2 B_P B_D^f L_\sigma \left\| \boldsymbol{P}_{full}^{(l+1),m} \boldsymbol{H}_{local}^{(l),m} - \boldsymbol{P}_{full}^{(l+1),m} \boldsymbol{H}_{full}^{(l),m} \right\|_F \\
&\overset{(d)}{\leq} B_W B_D^l C_\sigma \left\| \boldsymbol{P}_{local}^{(l+1),m} - \boldsymbol{P}_{full}^{(l+1),m} \right\|_F + B_W B_P C\sigma \left\| \boldsymbol{D}_{local}^{(l+1),m} - \boldsymbol{D}_{full}^{(l+1),m} \right\|_F \\
&+ B_W^2 B_P B_D^f L_\sigma B_H^l \left\| \boldsymbol{P}_{local}^{(l+1),m} - \boldsymbol{P}_{full}^{(l+1),m} \right\|_F + B_W^2 B_P^2 B_D^f L_\sigma \left\| \boldsymbol{H}_{local}^{(l),m} - \boldsymbol{H}_{full}^{(l),m} \right\|_F \\
&\overset{(e)}{\leq} B_W B_D^l C_\sigma B_P + B_W^2 B_P^2 B_D^f L_\sigma B_H^l + B_W^2 B_P^2 B_D^f L_\sigma B_{\Delta H}^r \\
&+ B_W B_P C\sigma \left\| \boldsymbol{D}_{local}^{(l+1),m} - \boldsymbol{D}_{full}^{(l+1),m} \right\|_F \\
&\leq \left(B_W B_D^l C_\sigma B_P + B_W^2 B_P^2 B_D^f L_\sigma B_H^l + B_W^2 B_P^2 B_D^f L_\sigma B_{\Delta H}^r \right)^{L-l} \\
&+ \left(B_W B_P C\sigma \right)^{L-l} \left\| \boldsymbol{D}_{local}^{(L),m} - \boldsymbol{D}_{full}^{(L),m} \right\|_F \\
&\overset{(f)}{\leq} \left(B_W B_D^l C_\sigma B_P + B_W^2 B_P^2 B_D^f L_\sigma B_H^l + B_W^2 B_P^2 B_D^f L_\sigma B_{\Delta H}^r \right)^{L-l} \\
&+ \left(B_W B_P C\sigma \right)^{L-l} L_l \left\| \boldsymbol{H}_{local}^{(L),m} - \boldsymbol{H}_{full}^{(L),m} \right\|_F \\
&\overset{(g)}{\leq} \left(B_W B_D^l C_\sigma B_P + B_W^2 B_P^2 B_D^f L_\sigma B_H^l + B_W^2 B_P^2 B_D^f L_\sigma B_{\Delta H}^r \right)^{L-l} \\
&+ \left(B_W B_P C\sigma \right)^{L-l} L_l B_{\Delta H}^r \\
&\leq \max_{1 \leq l \leq L} \Bigg(\left(B_W B_D^l C_\sigma B_P + B_W^2 B_P^2 B_D^f L_\sigma B_H^l + B_W^2 B_P^2 B_D^f L_\sigma B_{\Delta H}^r \right)^{L-l} \\
&+ \left(B_W B_P C\sigma \right)^{L-l} L_l B_{\Delta H}^r \Bigg),
\end{aligned}
$$

where (a) follows from Assumption 5.3, (b) uses Assumptions 5.2 and 5.3 and Lemma D.3, (c) is because of Assumptions 5.2 and 5.3, (d) results from Assumption 5.3 and Lemma D.3, (e) is due to Assumption 5.3 and Eq. (14), (f) utilizes Assumption 5.1, and (g) uses Eq. (14).

□

D.6 Errors of Gradients

Lemma D.6. *Under Assumptions 5.1–5.3, the errors between the stochastic gradients and the full gradients are bounded as follows:*

$$
\left\| \nabla F_{local}^m \left(\boldsymbol{\theta}^m \right) - \nabla \widetilde{F}_{local}^m \left(\boldsymbol{\theta}^m \right) \right\|_F \leq L B_{\Delta G}^l, \quad \left\| \nabla F_{full}^m \left(\boldsymbol{\theta}^m \right) - \nabla \widetilde{F}_{full}^m \left(\boldsymbol{\theta}^m \right) \right\|_F \leq L B_{\Delta G}^f,
$$

where

$$
\begin{aligned}
B_{\Delta G}^l = \max_{1 \leq l \leq L} \Bigg(& \left(B_D^l C_\sigma + B_P B_H^l B_D^l L_\sigma B_W \right) B_H^l B_{\Delta P}^l + B_P B_H^l C_\sigma B_{\Delta D}^l \\
& + \left(B_D^l C_\sigma + B_P B_H^l B_D^l L_\sigma B_W \right) B_P B_{\Delta H}^l \Bigg), \qquad (15) \\
B_{\Delta G}^f = \max_{1 \leq l \leq L} \Bigg(& \left(B_D^f C_\sigma + B_P B_H^f B_D^f L_\sigma B_W \right) B_H^f B_{\Delta P}^f + B_P B_H^f C_\sigma B_{\Delta D}^f \\
& + \left(B_D^f C_\sigma + B_P B_H^f B_D^f L_\sigma B_W \right) B_P B_{\Delta H}^f \Bigg) \qquad (16)
\end{aligned}
$$

Proof.

$$
\begin{aligned}
&\left\|\widetilde{\boldsymbol{G}}_{local}^{(l),m}-\boldsymbol{G}_{local}^{(l),m}\right\|_F \\
&=\left\|\left[\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}\right]^\top\widetilde{\boldsymbol{D}}_{local}^{(l),m}\circ\nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{local}^{(l),m}\right)-\left[\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}\right]^\top\boldsymbol{D}_{local}^{(l),m}\circ\nabla\sigma\left(\boldsymbol{Z}_{local}^{(l),m}\right)\right\|_F \\
&\leq\left\|\left[\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}\right]^\top\widetilde{\boldsymbol{D}}_{local}^{(l),m}\circ\nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{local}^{(l),m}\right)-\left[\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}\right]^\top\widetilde{\boldsymbol{D}}_{local}^{(l),m}\circ\nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{local}^{(l),m}\right)\right\|_F \\
&+\left\|\left[\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}\right]^\top\widetilde{\boldsymbol{D}}_{local}^{(l),m}\circ\nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{local}^{(l),m}\right)-\left[\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}\right]^\top\boldsymbol{D}_{local}^{(l),m}\circ\nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{local}^{(l),m}\right)\right\|_F \\
&+\left\|\left[\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}\right]^\top\boldsymbol{D}_{local}^{(l),m}\circ\nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{local}^{(l),m}\right)-\left[\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}\right]^\top\boldsymbol{D}_{local}^{(l),m}\circ\nabla\sigma\left(\boldsymbol{Z}_{local}^{(l),m}\right)\right\|_F \\
&\overset{(a)}{\leq}B_D^lC_\sigma\left\|\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}-\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}\right\|_F+B_PB_H^lC_\sigma\left\|\widetilde{\boldsymbol{D}}_{local}^{(l),m}-\boldsymbol{D}_{local}^{(l),m}\right\|_F \\
&+B_PB_H^lB_D^l\left\|\nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{local}^{(l),m}\right)-\nabla\sigma\left(\boldsymbol{Z}_{local}^{(l),m}\right)\right\|_F \\
&\overset{(b)}{\leq}B_D^lC_\sigma\left\|\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}-\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}\right\|_F+B_PB_H^lC_\sigma\left\|\widetilde{\boldsymbol{D}}_{local}^{(l),m}-\boldsymbol{D}_{local}^{(l),m}\right\|_F \\
&+B_PB_H^lB_D^lL_\sigma B_W\left\|\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}-\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}\right\|_F \\
&\leq\left(B_D^lC_\sigma+B_PB_H^lB_D^lL_\sigma B_W\right)\left\|\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}-\boldsymbol{P}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}\right\|_F \\
&+\left(B_D^lC_\sigma+B_PB_H^lB_D^lL_\sigma B_W\right)\left\|\boldsymbol{P}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}-\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}\right\|_F \\
&+B_PB_H^lC_\sigma\left\|\widetilde{\boldsymbol{D}}_{local}^{(l),m}-\boldsymbol{D}_{local}^{(l),m}\right\|_F \\
&\overset{(c)}{\leq}\left(B_D^lC_\sigma+B_PB_H^lB_D^lL_\sigma B_W\right)B_H^l\left\|\widetilde{\boldsymbol{P}}_{local}^{(l),m}-\boldsymbol{P}_{local}^{(l),m}\right\|_F+B_PB_H^lC_\sigma\left\|\widetilde{\boldsymbol{D}}_{local}^{(l),m}-\boldsymbol{D}_{local}^{(l),m}\right\|_F \\
&+\left(B_D^lC_\sigma+B_PB_H^lB_D^lL_\sigma B_W\right)B_P\left\|\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}-\boldsymbol{H}_{local}^{(l-1),m}\right\|_F \\
&\overset{(d)}{\leq}\left(B_D^lC_\sigma+B_PB_H^lB_D^lL_\sigma B_W\right)B_H^lB_{\Delta P}^l+B_PB_H^lC_\sigma B_{\Delta D}^l \\
&+\left(B_D^lC_\sigma+B_PB_H^lB_D^lL_\sigma B_W\right)B_PB_{\Delta H}^l \\
&\leq\max_{1\leq l\leq L}\Bigg(\left(B_D^lC_\sigma+B_PB_H^lB_D^lL_\sigma B_W\right)B_H^lB_{\Delta P}^l+B_PB_H^lC_\sigma B_{\Delta D}^l \\
&+\left(B_D^lC_\sigma+B_PB_H^lB_D^lL_\sigma B_W\right)B_PB_{\Delta H}^l\Bigg):=B_{\Delta G}^l,
\end{aligned}
$$

where (a) follows from Assumptions 5.2 and 5.3 and Lemma D.3, (b) is because of Assumptions 5.2 and 5.3, (c) uses Assumption 5.3 and Lemma D.3, and (d) results from Lemma D.4 and Proposition D.1.

When client m performs local training with only its local data, the error between the stochastic gradient and the full-gradient can be bounded as:

$$
\left\|\nabla F_{local}^m\left(\boldsymbol{\theta}^m\right)-\nabla\widetilde{F}_{local}^m\left(\boldsymbol{\theta}^m\right)\right\|_F=\sum_{l=1}^L\left\|\boldsymbol{G}_{local}^{(l),m}-\widetilde{\boldsymbol{G}}_{local}^{(l),m}\right\|_F\leq LB_{\Delta G}^l.
$$

$$
\begin{aligned}
&\left\|\widetilde{\boldsymbol{G}}_{full}^{(l),m}-\boldsymbol{G}_{full}^{(l),m}\right\|_F \\
&=\Bigg\|\left[\widetilde{\boldsymbol{P}}_{local}^{(l),m}\widetilde{\boldsymbol{H}}_{local}^{(l-1),m}+\widetilde{\boldsymbol{P}}_{remote}^{(l),m}\widetilde{\boldsymbol{H}}_{remote}^{(l-1),m}\right]^\top\widetilde{\boldsymbol{D}}_{full}^{(l),m}\circ\nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l),m}\right) \\
&-\left[\boldsymbol{P}_{full}^{(l),m}\boldsymbol{H}_{full}^{(l-1),m}\right]^\top\boldsymbol{D}_{full}^{(l),m}\circ\nabla\sigma\left(\boldsymbol{Z}_{full}^{(l),m}\right)\Bigg\|_F
\end{aligned}
$$

$$
\begin{aligned}
&\leq \left\| \left[\widetilde{\boldsymbol{P}}_{full}^{(l),m} \widetilde{\boldsymbol{H}}_{full}^{(l-1),m}\right]^\top \widetilde{\boldsymbol{D}}_{full}^{(l),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l),m}\right) - \left[\boldsymbol{P}_{full}^{(l),m} \boldsymbol{H}_{full}^{(l-1),m}\right]^\top \widetilde{\boldsymbol{D}}_{full}^{(l),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l),m}\right) \right\|_F \\
&+ \left\| \left[\boldsymbol{P}_{full}^{(l),m} \boldsymbol{H}_{full}^{(l-1),m}\right]^\top \widetilde{\boldsymbol{D}}_{full}^{(l),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l),m}\right) - \left[\boldsymbol{P}_{full}^{(l),m} \boldsymbol{H}_{full}^{(l-1),m}\right]^\top \boldsymbol{D}_{full}^{(l),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l),m}\right) \right\|_F \\
&+ \left\| \left[\boldsymbol{P}_{full}^{(l),m} \boldsymbol{H}_{full}^{(l-1),m}\right]^\top \boldsymbol{D}_{full}^{(l),m} \circ \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l),m}\right) - \left[\boldsymbol{P}_{full}^{(l),m} \boldsymbol{H}_{full}^{(l-1),m}\right]^\top \boldsymbol{D}_{full}^{(l),m} \circ \nabla\sigma\left(\boldsymbol{Z}_{full}^{(l),m}\right) \right\|_F \\
&\overset{(a)}{\leq} B_D^f C_\sigma \left\| \widetilde{\boldsymbol{P}}_{full}^{(l),m} \widetilde{\boldsymbol{H}}_{full}^{(l-1),m} - \boldsymbol{P}_{full}^{(l),m} \boldsymbol{H}_{full}^{(l-1),m} \right\|_F + B_P B_H^f C_\sigma \left\| \widetilde{\boldsymbol{D}}_{full}^{(l),m} - \boldsymbol{D}_{full}^{(l),m} \right\|_F \\
&+ B_P B_H^f B_D^f \left\| \nabla\sigma\left(\widetilde{\boldsymbol{Z}}_{full}^{(l),m}\right) - \nabla\sigma\left(\boldsymbol{Z}_{full}^{(l),m}\right) \right\|_F \\
&\overset{(b)}{\leq} B_D^f C_\sigma \left\| \widetilde{\boldsymbol{P}}_{full}^{(l),m} \widetilde{\boldsymbol{H}}_{full}^{(l-1),m} - \boldsymbol{P}_{full}^{(l),m} \boldsymbol{H}_{full}^{(l-1),m} \right\|_F + B_P B_H^f C_\sigma \left\| \widetilde{\boldsymbol{D}}_{full}^{(l),m} - \boldsymbol{D}_{full}^{(l),m} \right\|_F \\
&+ B_P B_H^f B_D^f L_\sigma B_W \left\| \widetilde{\boldsymbol{P}}_{full}^{(l),m} \widetilde{\boldsymbol{H}}_{full}^{(l-1),m} - \boldsymbol{P}_{full}^{(l),m} \boldsymbol{H}_{full}^{(l-1),m} \right\|_F \\
&\leq \left(B_D^f C_\sigma + B_P B_H^f B_D^f L_\sigma B_W\right) \left\| \widetilde{\boldsymbol{P}}_{full}^{(l),m} \widetilde{\boldsymbol{H}}_{full}^{(l-1),m} - \boldsymbol{P}_{full}^{(l),m} \widetilde{\boldsymbol{H}}_{full}^{(l-1),m} \right\|_F \\
&+ \left(B_D^f C_\sigma + B_P B_H^f B_D^f L_\sigma B_W\right) \left\| \boldsymbol{P}_{full}^{(l),m} \widetilde{\boldsymbol{H}}_{full}^{(l-1),m} - \boldsymbol{P}_{full}^{(l),m} \boldsymbol{H}_{full}^{(l-1),m} \right\|_F \\
&+ B_P B_H^f C_\sigma \left\| \widetilde{\boldsymbol{D}}_{full}^{(l),m} - \boldsymbol{D}_{full}^{(l),m} \right\|_F \\
&\overset{(c)}{\leq} \left(B_D^f C_\sigma + B_P B_H^f B_D^f L_\sigma B_W\right) B_H^f \left\| \widetilde{\boldsymbol{P}}_{full}^{(l),m} - \boldsymbol{P}_{full}^{(l),m} \right\|_F + B_P B_H^f C_\sigma \left\| \widetilde{\boldsymbol{D}}_{full}^{(l),m} - \boldsymbol{D}_{full}^{(l),m} \right\|_F \\
&+ \left(B_D^f C_\sigma + B_P B_H^f B_D^f L_\sigma B_W\right) B_P \left\| \widetilde{\boldsymbol{H}}_{full}^{(l-1),m} - \boldsymbol{H}_{full}^{(l-1),m} \right\|_F \\
&\overset{(d)}{\leq} \left(B_D^f C_\sigma + B_P B_H^f B_D^f L_\sigma B_W\right) B_H^f B_{\Delta P}^f + B_P B_H^f C_\sigma B_{\Delta D}^f \\
&+ \left(B_D^f C_\sigma + B_P B_H^f B_D^f L_\sigma B_W\right) B_P B_{\Delta H}^f \\
&\leq \max_{1\leq l\leq L} \Bigg(\left(B_D^f C_\sigma + B_P B_H^f B_D^f L_\sigma B_W\right) B_H^f B_{\Delta P}^f + B_P B_H^f C_\sigma B_{\Delta D}^f \\
&+ \left(B_D^f C_\sigma + B_P B_H^f B_D^f L_\sigma B_W\right) B_P B_{\Delta H}^f \Bigg) := B_{\Delta G}^f,
\end{aligned}
$$

where (a) results from Assumptions 5.2 and 5.3 and Lemma D.3, (b) uses Assumptions 5.2 and 5.3, (c) is due to Assumption 5.3 and Lemma D.3, and (d) is because of Lemma D.4 and Proposition D.1.

When client m conducts cross-client training using its local data and the cross-client neighbors, the error between the stochastic gradient and the full-gradient can be bounded as:

$$
\left\| \nabla F_{full}^m\left(\boldsymbol{\theta}^m\right) - \nabla \widetilde{F}_{full}^m\left(\boldsymbol{\theta}^m\right) \right\|_F = \sum_{l=1}^{L} \left\| \boldsymbol{G}_{full}^{(l),m} - \widetilde{\boldsymbol{G}}_{full}^{(l),m} \right\|_F \leq L B_{\Delta G}^f.
$$

□

Lemma D.7. *Under Assumptions 5.1–5.3, the error between the full gradient computed with both the local graph data and the cross-client neighbors and the full gradient computed with only the local graph data is upper-bounded as follows:*

$$
\left\| \nabla F_{full}^m\left(\boldsymbol{\theta}^m\right) - \nabla F_{local}^m\left(\boldsymbol{\theta}^m\right) \right\|_F \leq L B_{\Delta G}^r,
$$

where

$$
\begin{aligned}
B_{\Delta G}^r = \max_{1\leq l\leq L} \Bigg(&\left(B_D^l C_\sigma + B_P B_H^f B_D^f L_\sigma B_W\right) B_P B_{\Delta H}^r + B_P B_H^f C_\sigma B_{\Delta D}^r \\
&+ \left(B_D^l C_\sigma + B_P B_H^f B_D^f L_\sigma B_W\right) B_H^f B_P \Bigg)
\end{aligned} \tag{17}
$$

Proof.

$$
\begin{aligned}
&\left\|\boldsymbol{G}_{local}^{(l),m}-\boldsymbol{G}_{full}^{(l),m}\right\|_F \\
&=\left\|\left[\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}\right]^\top \boldsymbol{D}_{local}^{(l),m}\circ\nabla\sigma\left(\boldsymbol{Z}_{local}^{(l),m}\right)-\left[\boldsymbol{P}_{full}^{(l),m}\boldsymbol{H}_{full}^{(l-1),m}\right]^\top \boldsymbol{D}_{full}^{(l),m}\circ\nabla\sigma\left(\boldsymbol{Z}_{full}^{(l),m}\right)\right\|_F \\
&\leq\left\|\left[\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}\right]^\top \boldsymbol{D}_{local}^{(l),m}\circ\nabla\sigma\left(\boldsymbol{Z}_{local}^{(l),m}\right)-\left[\boldsymbol{P}_{full}^{(l),m}\boldsymbol{H}_{full}^{(l-1),m}\right]^\top \boldsymbol{D}_{local}^{(l),m}\circ\nabla\sigma\left(\boldsymbol{Z}_{local}^{(l),m}\right)\right\|_F \\
&+\left\|\left[\boldsymbol{P}_{full}^{(l),m}\boldsymbol{H}_{full}^{(l-1),m}\right]^\top \boldsymbol{D}_{local}^{(l),m}\circ\nabla\sigma\left(\boldsymbol{Z}_{local}^{(l),m}\right)-\left[\boldsymbol{P}_{full}^{(l),m}\boldsymbol{H}_{full}^{(l-1),m}\right]^\top \boldsymbol{D}_{full}^{(l),m}\circ\nabla\sigma\left(\boldsymbol{Z}_{local}^{(l),m}\right)\right\|_F \\
&+\left\|\left[\boldsymbol{P}_{full}^{(l),m}\boldsymbol{H}_{full}^{(l-1),m}\right]^\top \boldsymbol{D}_{full}^{(l),m}\circ\nabla\sigma\left(\boldsymbol{Z}_{local}^{(l),m}\right)-\left[\boldsymbol{P}_{full}^{(l),m}\boldsymbol{H}_{full}^{(l-1),m}\right]^\top \boldsymbol{D}_{full}^{(l),m}\circ\nabla\sigma\left(\boldsymbol{Z}_{full}^{(l),m}\right)\right\|_F \\
&\overset{(a)}{\leq} B_D^l C_\sigma\left\|\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}-\boldsymbol{P}_{full}^{(l),m}\boldsymbol{H}_{full}^{(l-1),m}\right\|_F+B_P B_H^f C_\sigma\left\|\boldsymbol{D}_{local}^{(l),m}-\boldsymbol{D}_{full}^{(l),m}\right\|_F \\
&+B_P B_H^f B_D^f\left\|\nabla\sigma\left(\boldsymbol{Z}_{local}^{(l),m}\right)-\nabla\sigma\left(\boldsymbol{Z}_{full}^{(l),m}\right)\right\|_F \\
&\overset{(b)}{\leq} B_D^l C_\sigma\left\|\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}-\boldsymbol{P}_{full}^{(l),m}\boldsymbol{H}_{full}^{(l-1),m}\right\|_F+B_P B_H^f C_\sigma\left\|\boldsymbol{D}_{local}^{(l),m}-\boldsymbol{D}_{full}^{(l),m}\right\|_F \\
&+B_P B_H^f B_D^f L_\sigma B_W\left\|\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}-\boldsymbol{P}_{full}^{(l),m}\boldsymbol{H}_{full}^{(l-1),m}\right\|_F \\
&\leq\left(B_D^l C_\sigma+B_P B_H^f B_D^f L_\sigma B_W\right)\left\|\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{local}^{(l-1),m}-\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{full}^{(l-1),m}\right\|_F \\
&+\left(B_D^l C_\sigma+B_P B_H^f B_D^f L_\sigma B_W\right)\left\|\boldsymbol{P}_{local}^{(l),m}\boldsymbol{H}_{full}^{(l-1),m}-\boldsymbol{P}_{full}^{(l),m}\boldsymbol{H}_{full}^{(l-1),m}\right\|_F \\
&+B_P B_H^f C_\sigma\left\|\boldsymbol{D}_{local}^{(l),m}-\boldsymbol{D}_{full}^{(l),m}\right\|_F \\
&\overset{(c)}{\leq}\left(B_D^l C_\sigma+B_P B_H^f B_D^f L_\sigma B_W\right)B_P\left\|\boldsymbol{H}_{local}^{(l-1),m}-\boldsymbol{H}_{full}^{(l-1),m}\right\|_F \\
&+\left(B_D^l C_\sigma+B_P B_H^f B_D^f L_\sigma B_W\right)B_H^f\left\|\boldsymbol{P}_{local}^{(l),m}-\boldsymbol{P}_{full}^{(l),m}\right\|_F+B_P B_H^f C_\sigma\left\|\boldsymbol{D}_{local}^{(l),m}-\boldsymbol{D}_{full}^{(l),m}\right\|_F \\
&\overset{(d)}{\leq}\left(B_D^l C_\sigma+B_P B_H^f B_D^f L_\sigma B_W\right)B_P B_{\Delta H}^r+B_P B_H^f C_\sigma B_{\Delta D}^r \\
&+\left(B_D^l C_\sigma+B_P B_H^f B_D^f L_\sigma B_W\right)B_H^f B_P \\
&\leq\max_{1\leq l\leq L}\Bigg(\left(B_D^l C_\sigma+B_P B_H^f B_D^f L_\sigma B_W\right)B_P B_{\Delta H}^r+B_P B_H^f C_\sigma B_{\Delta D}^r \\
&+\left(B_D^l C_\sigma+B_P B_H^f B_D^f L_\sigma B_W\right)B_H^f B_P\Bigg)=B_{\Delta G}^r,
\end{aligned}
$$

where (a) is because of Assumptions 5.2 and 5.3 and Lemma D.3, (b) uses Assumptions 5.2 and 5.3, (c) follow from Assumption 5.3 and Lemma D.3, and (d) results from Assumption 5.3 and Lemma D.5.

The error between the full gradient computed with both the local graph data and the cross-client neighbors and the full gradient computed with only the local graph data is bounded as follows:

$$
\left\|\nabla F_{full}^m\left(\boldsymbol{\theta}^m\right)-\nabla F_{local}^m\left(\boldsymbol{\theta}^m\right)\right\|_F=\sum_{l=1}^{L}\left\|\boldsymbol{G}_{local}^{(l),m}-\boldsymbol{G}_{full}^{(l),m}\right\|_F\leq LB_{\Delta G}^r.
$$

□

D.7 Main Proof of Theorem 5.6

Theorem D.8. *Under Assumptions 5.1–5.3, choose step-size* $\alpha=\min\left\{\sqrt{M}/\sqrt{T},1/L_F\right\}$*, where* L_F *is the smoothness constant given in Lemma D.2. The output of* Swift-FedGNN *with a* L*-layer*

GNN satisfies:

$$\frac{1}{T}\sum_{t=0}^{T-1}\left\|\nabla\mathcal{L}\left(\boldsymbol{\theta}_t\right)\right\|^2 \leq \frac{2}{\sqrt{MT}}\left(\mathcal{L}\left(\boldsymbol{\theta}_0\right)-\mathcal{L}\left(\boldsymbol{\theta}^*\right)\right)+\left(1-\frac{K}{IM}\right)L^2\left(B_{\Delta G}^l+B_{\Delta G}^r\right)^2+\frac{K}{IM}L^2(B_{\Delta G}^f)^2.$$

Proof.

$$\begin{aligned}
&\mathcal{L}\left(\boldsymbol{\theta}_{t+1}\right)-\mathcal{L}\left(\boldsymbol{\theta}_t\right)\\
&\overset{(a)}{\leq}\left\langle\nabla\mathcal{L}\left(\boldsymbol{\theta}_t\right),\boldsymbol{\theta}_{t+1}-\boldsymbol{\theta}_t\right\rangle+\frac{L_F}{2}\left\|\boldsymbol{\theta}_{t+1}-\boldsymbol{\theta}_t\right\|^2\\
&\overset{(b)}{=}-\alpha\left\langle\nabla\mathcal{L}\left(\boldsymbol{\theta}_t\right),\frac{1}{M}\sum_{m\in\mathcal{M}}\nabla\widetilde{F}^m\left(\boldsymbol{\theta}_t^m\right)\right\rangle+\frac{L_F}{2}\alpha^2\left\|\frac{1}{M}\sum_{m\in\mathcal{M}}\nabla\widetilde{F}^m\left(\boldsymbol{\theta}_t^m\right)\right\|^2\\
&\overset{(c)}{=}-\frac{\alpha}{2}\left\|\nabla\mathcal{L}\left(\boldsymbol{\theta}_t\right)\right\|^2-\frac{\alpha}{2}\left\|\frac{1}{M}\sum_{m\in\mathcal{M}}\nabla\widetilde{F}^m\left(\boldsymbol{\theta}_t^m\right)\right\|^2+\frac{\alpha}{2}\left\|\nabla\mathcal{L}\left(\boldsymbol{\theta}_t\right)-\frac{1}{M}\sum_{m\in\mathcal{M}}\nabla\widetilde{F}^m\left(\boldsymbol{\theta}_t^m\right)\right\|^2\\
&+\frac{L_F}{2}\alpha^2\left\|\frac{1}{M}\sum_{m\in\mathcal{M}}\nabla\widetilde{F}^m\left(\boldsymbol{\theta}_t^m\right)\right\|^2\\
&=-\frac{\alpha}{2}\left\|\nabla\mathcal{L}\left(\boldsymbol{\theta}_t\right)\right\|^2-\frac{\alpha}{2}\left\|\frac{1}{M}\sum_{m\in\mathcal{M}}\nabla\widetilde{F}^m\left(\boldsymbol{\theta}_t^m\right)\right\|^2+\frac{\alpha}{2}\left\|\frac{1}{M}\sum_{m\in\mathcal{M}}\left(\nabla F^m\left(\boldsymbol{\theta}_t^m\right)-\nabla\widetilde{F}^m\left(\boldsymbol{\theta}_t^m\right)\right)\right\|^2\\
&+\frac{L_F}{2}\alpha^2\left\|\frac{1}{M}\sum_{m\in\mathcal{M}}\nabla\widetilde{F}^m\left(\boldsymbol{\theta}_t^m\right)\right\|^2\\
&\overset{(d)}{\leq}-\frac{\alpha}{2}\left\|\nabla\mathcal{L}\left(\boldsymbol{\theta}_t\right)\right\|^2-\frac{\alpha}{2}\left\|\frac{1}{M}\sum_{m\in\mathcal{M}}\nabla\widetilde{F}^m\left(\boldsymbol{\theta}_t^m\right)\right\|^2+\frac{\alpha}{2}\frac{1}{M}\sum_{m\in\mathcal{M}}\left\|\nabla F^m\left(\boldsymbol{\theta}_t^m\right)-\nabla\widetilde{F}^m\left(\boldsymbol{\theta}_t^m\right)\right\|^2\\
&+\frac{L_F}{2}\alpha^2\left\|\frac{1}{M}\sum_{m\in\mathcal{M}}\nabla\widetilde{F}^m\left(\boldsymbol{\theta}_t^m\right)\right\|^2\\
&\overset{(e)}{\leq}-\frac{\alpha}{2}\left\|\nabla\mathcal{L}\left(\boldsymbol{\theta}_t\right)\right\|^2+\frac{\alpha}{2}\frac{1}{M}\sum_{m\in\mathcal{M}}\left\|\nabla F^m\left(\boldsymbol{\theta}_t^m\right)-\nabla\widetilde{F}^m\left(\boldsymbol{\theta}_t^m\right)\right\|^2,
\end{aligned}\tag{18}$$

where (a) follows from Lemma D.2, (b) is because of the update rule in Swift-FedGNN, (c) uses $\langle\boldsymbol{x},\boldsymbol{y}\rangle=\frac{1}{2}\|\boldsymbol{x}\|^2+\frac{1}{2}\|\boldsymbol{y}\|^2-\frac{1}{2}\|\boldsymbol{x}-\boldsymbol{y}\|^2$, (d) utilizes $\|\sum_{i=1}^n\boldsymbol{x}_i\|^2\leq n\sum_{i=1}^n\|\boldsymbol{x}_i\|^2$, and (e) is due to the choice of $\alpha\leq 1/L_F$.

When $t\in[(n_t-1)I+1,n_tI-1]\cap\mathbb{Z}$, where $n_t=\{1,2,\cdots\}$, Swift-FedGNN conducts local training for all clients $m\in\mathcal{M}$. Thus,

$$\begin{aligned}
\left\|\nabla F^m\left(\boldsymbol{\theta}_t^m\right)-\nabla\widetilde{F}^m\left(\boldsymbol{\theta}_t^m\right)\right\| &= \left\|\nabla F_{full}^m\left(\boldsymbol{\theta}_t^m\right)-\nabla\widetilde{F}_{local}^m\left(\boldsymbol{\theta}_t^m\right)\right\|\\
&\leq\left\|\nabla F_{full}^m\left(\boldsymbol{\theta}_t^m\right)-\nabla F_{local}^m\left(\boldsymbol{\theta}_t^m\right)\right\|+\left\|\nabla F_{local}^m\left(\boldsymbol{\theta}_t^m\right)-\nabla\widetilde{F}_{local}^m\left(\boldsymbol{\theta}_t^m\right)\right\|\\
&\overset{(a)}{\leq}LB_{\Delta G}^r+LB_{\Delta G}^l,
\end{aligned}\tag{19}$$

where (a) follows from Lemmas D.6 and D.7.

When $t=n_tI$, where $n_t=\{1,2,\cdots\}$, Swift-FedGNN performs local training for clients $m\in\mathcal{M}\backslash\mathcal{K}$, and thus the inequality (19) holds for these clients. The randomly sampled clients $m\in\mathcal{K}$ conduct cross-client training, and thus

$$\left\|\nabla F^m\left(\boldsymbol{\theta}_t^m\right)-\nabla\widetilde{F}^m\left(\boldsymbol{\theta}_t^m\right)\right\|=\left\|\nabla F_{full}^m\left(\boldsymbol{\theta}_t^m\right)-\nabla\widetilde{F}_{full}^m\left(\boldsymbol{\theta}_t^m\right)\right\|\overset{(a)}{\leq}LB_{\Delta G}^f,$$

where (a) uses Lemma D.6.

Telescoping (18) from $i = (n_t - 1)\, I + 1$ to $n_t I$, we have

$$\begin{aligned}
&\sum_{i=(n_t-1)I+1}^{n_t I} \left(\mathcal{L}\left(\boldsymbol{\theta}_{i+1}\right) - \mathcal{L}\left(\boldsymbol{\theta}_i\right)\right) \\
&\leq -\frac{\alpha}{2} \sum_{i=(n_t-1)I+1}^{n_t I} \left\| \nabla \mathcal{L}\left(\boldsymbol{\theta}_i\right) \right\|^2 + \frac{\alpha}{2}(I-1)L^2 \left(B^l_{\Delta G} + B^r_{\Delta G}\right)^2 + \frac{\alpha}{2M} K L^2 \left(B^f_{\Delta G}\right)^2 \\
&+ \frac{\alpha}{2M}(M-K)L^2 \left(B^l_{\Delta G} + B^r_{\Delta G}\right)^2 .
\end{aligned}$$

Choosing $T = n_t I$ yields

$$\begin{aligned}
&\sum_{t=0}^{T-1} \left(\mathcal{L}\left(\boldsymbol{\theta}_{t+1}\right) - \mathcal{L}\left(\boldsymbol{\theta}_t\right)\right) \\
&\leq -\frac{\alpha}{2} \sum_{t=0}^{T-1} \left\| \nabla \mathcal{L}\left(\boldsymbol{\theta}_t\right) \right\|^2 + \frac{\alpha}{2}(T-n_t)L^2 \left(B^l_{\Delta G} + B^r_{\Delta G}\right)^2 + n_t \frac{\alpha}{2M} K L^2 \left(B^f_{\Delta G}\right)^2 \\
&+ n_t \frac{\alpha}{2M}(M-K)L^2 \left(B^l_{\Delta G} + B^r_{\Delta G}\right)^2 .
\end{aligned}$$

Rearranging the terms and multiplying both sides by $2/\alpha$, we get

$$\begin{aligned}
&\sum_{t=0}^{T-1} \left\| \nabla \mathcal{L}\left(\boldsymbol{\theta}_t\right) \right\|^2 \\
&\leq \frac{2}{\alpha} \sum_{t=0}^{T-1} \left(\mathcal{L}\left(\boldsymbol{\theta}_t\right) - \mathcal{L}\left(\boldsymbol{\theta}_{t+1}\right)\right) + (T-n_t)L^2 \left(B^l_{\Delta G} + B^r_{\Delta G}\right)^2 + \frac{n_t}{M} K L^2 \left(B^f_{\Delta G}\right)^2 \\
&+ \frac{n_t}{M}(M-K)L^2 \left(B^l_{\Delta G} + B^r_{\Delta G}\right)^2 .
\end{aligned}$$

Dividing both sides by T and choosing $\alpha = \sqrt{M}/\sqrt{T}$ completes the proof of Theorem 5.6.

□